Universidad de Buenos Aires
Facultad de Ciencias Exactas y Naturales
Departamento de Computación

[your title here]

Tesis de Licenciatura en Ciencias de la Computación

Rafael Romani

Director: Santiago Cifuentes
Codirector: Santiago Figueira
Buenos Aires, 2026

[YOUR TITLE HERE]

[Acá va el resumen]

Palabras claves: [acá van las palabras clave]

[YOUR TITLE HERE]

[write your abstract here]

Keywords: [write your keywords here]

[acá van los agradecimientos]

CONTENTS

1. INTRODUCTION

1.1 Uso y poder de los LLMS

In 2017, a group of researchers working at Google published a paper introducing a technology that revolutionized society. In *Attention is All You Need*, they presented the transformers, today's core of large language models (LLMs) and generative AI.

This architecture was build intended as text sequence-to-sequence model for machine translation but today has applications in various fields as natural language processing, computer vision or speech recognition.

The first model implementing transformers open for public use was ChatGPT by OpenAI. It made its debut in November 2022 and by December 2024 it reported 300 million global weekly users. Now there are many models competing for this market as Claude, from Anthropic, or Gemini from Google.

These models can be all be grouped under the generative AI tools. This is a technology that is arguably the most rapid technology adopted by society. In the second half of 2025, roughly one in six people worldwide were using generative AI tools. not sure how to cite this https://www.microsoft.com/en-us/research/wp-content/uploads/2026/01/Microsoft-AI-Diffusion-Report-2025-H2.pdf

SF: Lo citás como misceláneous

LLMs are being used not only as chatbots for asking small questions, but are shaping the industry and future. For example, [6] found that more than 17% of the Computer Science's papers published between January 2020 and February 2024 on the *arXiv*, *bioRxiv*, and *Nature* portfolio journals had sentences heavily edited by ChatGPT.

Missing usage in industry

So naturally, the question about their computing power pops up. Are this kind of models really capable of doing the stuff for what they are being used?

- well-suited to parallelism enables transformer models to take full advantage of the power and speed offered by GPUs during both training and inference. This possibility, in turn, unlocked the opportunity to train transformer models on unprecedentedly massive datasets through self-supervised learning.
- chatgpt, claude, gemini,
- every day users
- number of users?
- use in the industry?
- other uses besides the chatbots?
- no usamos normalizacion porque andie lo usa, es complicado. Si se achican los bitsize empieza a tener menos sentido

cite this https://www.ibm.co model

1.2 Pero qué es un LLM?

1.3 Formalizaciones previas y resultados que ignoran los aspectos probabilísticos

1.4 Objetivos y contribuciones

2. THEORETICAL FRAMEWORK

2.1 Transformers

In this work we are going to analyze transformers as language recognizers. A transformer is a machine that works over an alphabet Σ. It has a single tape from which it reads the input and onto which it appends additional tokens in an autoregressive manner. There are two distinguished tokens called **decision symbols**. Once the transformer writes one of those, it halts. We say that a transformer accepts a word if, when it halts, the decision symbol printed is the accepting one (q_{accept}). Analogously, a transformer rejects a word if the decision symbol printed is the rejecting one (q_{reject}).

In each step, the transformer maps the content of the tape to a probability distribution over Σ. Then, depending on this distribution, the transformer prints the next token. The transformer iterates this process until it halts. These iterations are called **chain of thought**.

RR: Agregar una cita

We split the process of generating the next token into two separate steps. First, the computation of the probability distribution over the alphabet which we call distribution generation.

SF así como hiciste con cot, definí nuevos términos con un estilo especial. Yo uso defstlye como comando y después decido si es itálica, negrita, etc

RR defino distribution generation y token selection más abajo y los puse en negrita. Debería marcarlos también acá?

Second, the selection of a token based on the probability distribution, which we call token selection. In this work we analyze and compare two different algorithms for token selection.

Most prior work only considers transformers that operate over an infinite tape. This is far from the real models, thus the transformers studied in this thesis work over a finite tape. The size of the tape is defined, for each input size n, as $\text{budget}(n) + n$. It is important to remark that the size of the tape bounds the number of steps of CoT that the transformer makes. For an input word of length n, no computation can take more than $\text{budget}(n)$ steps because every step writes a token in the tape.

The process of distribution generation is dependent on many parameters. In the real models, these parameters are the ones calculated during the training.
We refer to them as θ and consider them as part of the definition of a transformer.

We denote by $\mathsf{TF}(\alpha)$ the token outputted after one step of the computation of the transformer TF when the tape content is $\alpha \in \Sigma^*$. The autoregressive token generation is defined recursively. The token outputted after i steps is $\mathsf{TF}^i(\alpha) := \mathsf{TF}^{i-1}(\alpha\mathsf{TF}(\alpha))$ with base case $\mathsf{TF}^1(\alpha) := \mathsf{TF}(\alpha)$. In other words, the output of the ith step is $\sigma_i = \mathsf{TF}^i(\alpha) = \mathsf{TF}(\alpha\sigma_1 \dots \sigma_{i-1})$ where $\sigma_j \in \Sigma$ for all j.
We denote by $\mathsf{TF}^*(\alpha)$ the token written by the transformer when it halts, i.e., at the end of the computation, when the input is α. As said before, the transformer only halts when it prints a decision symbol (q_{accept} or q_{reject}). Therefore, $\mathsf{TF}^*(\alpha) \in \{q_{accept}, q_{reject}\}$.

As stated before, the **distribution generation** is a mapping of the content of the tape to a probability distribution over Σ. There are different algorithms for this process in the

literature. All transformers considered in this work employ the one presented in [5]. This algorithm consists of four parts: a token embedding layer (TE), a position encoding layer (PE), an output linear layer (OUTPUT) and a stack of L identical decoder layers, where L is also called the depth of the model. Each decoder layer has two sub-layers: a multi-head self-attention layer (ATTN) and a position-wise fully-connected feed-forward network (FF). Each part and layer has its own trainable parameters. Then, we can define the distribution generation algorithm by its parameters $\theta = \{\theta_{\mathsf{TE}}, \theta_{\mathsf{PE}}, \{\theta^l_{\mathsf{ATTN}}, \theta^l_{\mathsf{FF}}\}_{l=0}^{L-1}, \theta_{\mathsf{OUTPUT}}\}$. The algorithm works over vectors of numbers of certain dimension d. The dimension is dependent on the size of the input word. We call $d(n)$ to the **embedding size** when the input word has size n.

Token Embedding: It is a mapping from Σ to vectors in $\mathbb{R}^d$. This mapping is defined by $\theta_{\mathsf{TE}} \in \mathbb{R}^{d \times |\Sigma|}$. For all $\sigma \in \Sigma$, we note $\theta_{\mathsf{TE}}(\sigma) \in \mathbb{R}^d$ to the mapping of token σ.

Position Encoding: It is a mapping from the positions of the tape to vectors in $\mathbb{R}^d$. This mapping is defined by $\theta_{\mathsf{PE}} \in \mathbb{R}^{d \times \text{budget}(n)+n}$. For all position of the tape $1 \leq i \leq \text{budget}(n) + n$ we note $\theta_{\mathsf{PE}}(i) \in \mathbb{R}^d$ to the mapping of position i.

SC Intuición de todas estas capas?

RR Lo hago en la intro? Me parece el mejor lugar para hacerlo

Self Attention Layer: It is a mapping from $(\mathbb{R}^d)^a$ to $(\mathbb{R}^d)^a$. Given parameters $\theta_{\mathsf{ATTN}} = (W_Q, W_K, W_V, W_O) \in \mathbb{R}^{d \times d} \times \mathbb{R}^{d \times d} \times \mathbb{R}^{d \times d} \times \mathbb{R}^{d \times d}$, the layer is defined by the algorithm 1. This layer is defined only as a single head attention layer because a multi-head attention layer with any constant number of heads can be simulated with multiple single-head attention layers [15].

Algorithm 1 Causal Self-Attention, ATTN

Input: Parameter $\theta_{\mathsf{ATTN}} = (W_Q, W_K, W_V, W_O)$ and embedding $h = (h_1, \ldots, h_a) \in \mathbb{R}^d \times \cdots \times \mathbb{R}^d$.

Output: Embedding $h' = (h'_1, \ldots, h'_a) := \mathsf{ATTN}_{\theta_{\mathsf{ATTN}}}(h_1, \ldots, h_a)$.

1: $q_i := W_Q h_i,\ k_i := W_K h_i,\ v_i := W_V h_i,\ \forall i \in [1, \ldots, a]$
2: $s_i := \text{softmax}(\langle q_i, k_1 \rangle, \ldots, \langle q_i, k_i \rangle) \| (0, \ldots, 0)$
3: $h'_i := W_O \sum_{j=1}^{a} (s_i)_j v_j$

Feed Forward Network: It is a mapping from $\mathbb{R}^d$ to $\mathbb{R}^d$. Given $\theta_{\mathsf{FF}} = (W_1, b_1, W_2, b_2) \in \mathbb{R}^{d \times d} \times \mathbb{R}^d \times \mathbb{R}^{d \times d} \times \mathbb{R}^d$, the layer is defined as $\mathsf{FF}_{\theta_{\mathsf{FF}}}(h) := W_2\ \text{relu}(W_1 h + b_1) + b_2$, where $\text{relu} : \mathbb{R}^d \to \mathbb{R}^d$ replaces each position x_i by $\max(0, x_i)$.

Output Layer: It is a mapping from $\mathbb{R}^d$ to a probability distribution over Σ. Given $\theta_{\mathsf{OUTPUT}} \in \mathbb{R}^{|\Sigma| \times d}$, we define it as $\mathsf{OUTPUT}_{\theta_{\mathsf{OUTPUT}}}(h) := \text{softmax}(\theta_{\mathsf{OUTPUT}} h)$.

All these parts come together for the distribution generator process in the algorithm 2.

The OUTPUT layer and the ATTN layer use a softmax function $\mathbb{R}^n$ to $\mathbb{R}^n$ defined as $\text{softmax}(x)_i = exp(x_i) / \sum_{j=1}^{n} exp(x_j)$.

The second component of a transformer is the **token selector**. This algorithm consumes the probability distribution computed by the distribution generator and returns a token.

In this work we study two selectors, we call them **deterministic selector** and **probabilistic selector**. Let $p \in \Delta(\Sigma)$ be the distribution over Σ outputted by the distribution

Algorithm 2 Distribution generator

Input: Transformer parameter $\theta = (\theta_{\mathsf{TE}}, \theta_{\mathsf{PE}}, \{\theta^l_{\mathsf{ATTN}}, \theta^l_{\mathsf{FF}}\}_{l=0}^{L-1}, \theta_{\mathsf{OUTPUT}})$ and tokens of the tape $\alpha \in \Sigma^*$.

Output: distribution $p_\theta(\cdot|\alpha)$.

1: $h_i^{(0)} \leftarrow \theta_{\mathsf{TE}}(\alpha_i) + \theta_{\mathsf{PE}}(i),\ \forall 1 \leq i \leq |\alpha|$
2: **for** $l = 0, \ldots, L-1$ **do**
3: $\quad (h_1^{(l+0.5)}, \ldots, h_{|\alpha|}^{(l+0.5)}) \leftarrow (h_1^{(l)}, \ldots, h_{|\alpha|}^{(l)}) + \mathsf{ATTN}_{\theta^{(l)}_{\mathsf{ATTN}}}(h_1^{(l)}, \ldots, h_{|\alpha|}^{(l)})$
4: $\quad h_i^{(l+1)} \leftarrow h_i^{(l+0.5)} + \mathsf{FF}_{\theta^{(l)}_{\mathsf{FF}}}(h_i^{(l+0.5)}),\ \forall 1 \leq i \leq |\alpha|$
5: **end for**
6: $p_\theta(\cdot|\alpha) \leftarrow \mathsf{OUTPUT}_{\theta_{\mathsf{OUTPUT}}}(h_{|\alpha|}^{(L)})$

generator. Then, the deterministic selector always returns the token with the largest probability (argmax over p). In transformers with this selector, we do not allow for two or more tokens to be tied with the maximum probability, i.e., $\nexists \sigma$ s.t. $p(\text{argmax}(p)) = p(\sigma) \wedge \sigma \neq \text{argmax}(p)$. In contrast, the probabilistic selector works non-deterministically. It returns a token $\sigma \in \Sigma$ with probability $p(\sigma)$. We allow for more than one maximum with this selector.

Although the presentation for transformers introduced in this section operates over real numbers, the transformers analyzed during this work utilize $\mathbb{F}_{e,s}$ for representing them. The formal construction is the same, but all operations over the set of real numbers are replaced but their corresponding in the set of $\mathbb{F}_{e,s}$ numbers.

RR Toyota tiene copiado lo mismo que está en esta sección en un apéndice donde hizo ctrl +f y reemplazo $\mathbb{R}$ por $\mathbb{F}_{e,s}$. Vale la pena que haga lo mismo?

Definition 2.1.1 (Deterministic transformer)**.** We call a transformer deterministic when the token selector is the deterministic selector. We say that a family of deterministic transformers $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ recognizes a language $\mathcal{L}$ when:

$$\alpha \in \mathcal{L} \iff \mathsf{TF}^*_{|\alpha|}(\alpha) = q_{accept}$$

As mentioned in Chapter 1, there is an unexplored area in the study of transformers expressiveness; a study of non-determinism is missing.

When TF is a deterministic transformer, $\mathsf{TF}^i(\alpha)$ is an exact symbol for all $\alpha \in \Sigma^*$ and $i \in \mathbb{N}$. On the contrary, when the token selection process of the transformer samples the distribution, it becomes a random variable. Therefore, we introduce the following definition.

Definition 2.1.2 (Probabilistic transformer)**.** We call probabilistic transformer to the transformers that use the probabilistic selector. We say that a family of probabilistic transformers $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ recognizes a language $\mathcal{L}$ when:

$$\alpha \in \mathcal{L} \iff \Pr[\mathsf{TF}^*_{|\alpha|}(\alpha) = q_{accept}] > 2/3$$

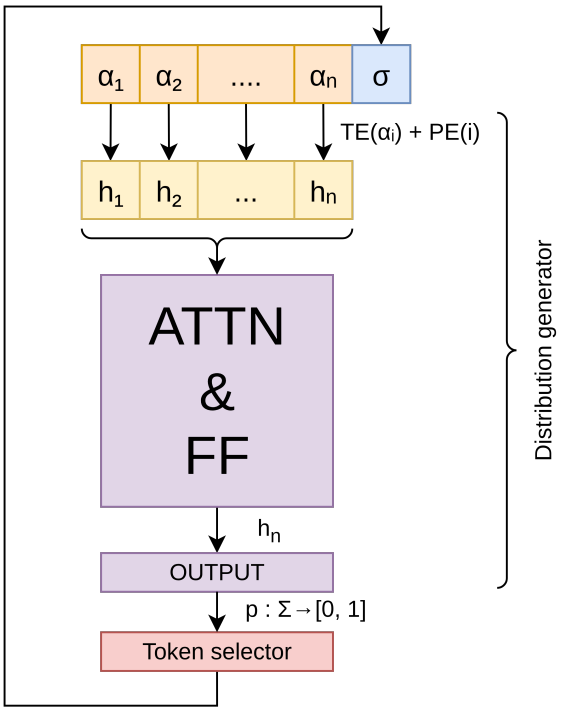

Fig. 2.1: Transformer architecture

2.2 Finite precision modeling

I'm missing good symbols for ∞ y $-\infty$.

In real models, the transformers typically work with 16- or 32-bit floating-point numbers, and there is active work in reducing their size. For this reason, in this thesis we follow the decision of [5] to work with constant-precision transformers, where the output of each arithmetic operation is rounded to the closest floating-point number representable by a fixed number of digits following IEEE 754 standard. This analysis avoids the unrealistic infinite precision assumption made in prior works [11]. The definitions stated in this section are taken from [5].

Intuitively, we represent the numbers as a tuple $\langle$ sign, significand, exponent$\rangle$ with a fixed number of bits for each coordinate. The tuple is interpreted as sign $\cdot$ significand $\cdot$ 2^{exponent}. For example, the number 8 is represented as $\langle +, 1, 3\rangle$. When fixing the number of bits for the significand and the exponent there are numbers non-representable. For example, with only 2 bits of significand and 3 of exponent, the number 9 can not be represented exactly. We can represent $8 = 1 \cdot 2^3$ and $12 = 3 \cdot 2^2$, thus when an arithmetic operation should output 9 we output 8.

Below we give a formal definition of the floating-point number and rounding operation. We use $\phi(a) = \sum_{i=1}^{k} 2^{k-i} a_i$ to denote the value of the binary number represented by $a \in \{0,1\}^k$ for any $k \in \mathbb{N}^+$.

Definition 2.2.1 (Floating-point Representation)**.** Let e be the number of bits for exponents and s be the number of bits for significand. A $(e + 2s + 1)$-bit binary string $a = (a_1, a_2, \ldots a_{e+2s+1}) \in \{0,1\}^{e+2s+1}$ is a floating-point binary representation of number $\phi_{e,s}(a) \triangleq \text{sign}(a) \cdot 2^{\text{exponent}(a)} \cdot \text{significand}(a)$ with e-bit exponent and $2s$-precision, where

the sign is $\text{sign}(a) \triangleq 2a_1 - 1$, the significand is $\text{significand}(a) \triangleq 2^{-s}\phi(a_{2:2s+1})$, and the exponent is $\text{exponent}(a) \triangleq \phi(a_{2s+2:2s+e+1}) - 2^{\max(0,e-1)}$. We define the set of numbers expressible with e bits of exponent and s bits of significand as $\mathbb{F}_{e,s}$. We define $\infty_{e,s}$ as the maximum number in $\mathbb{F}_{e,s}$. Similarly, we define $-\infty_{e,s}$ as the minimum. Since in this work we only analyze constant precision transformers, we will omit the subscripts of ∞ and $-\infty$.

Definition 2.2.2 (Correct Rounding)**.** For any $x \in \mathbb{R}$, we define correct rounding $\text{round}(x, \mathbb{F}_{e,s})$ as the closest number to x in $\mathbb{F}_{e,s}$. We break the tie by picking the one with a smaller absolute value. In particular, we denote the rounding operation with e-bit exponent, $2s$-bit precision by $\text{round}_{e,s}(\cdot) \triangleq \text{round}(\cdot, \mathbb{F}_{e,s})$, which is also denoted by $[\cdot]_{e,s}$ for convenience. We extend the definition of round and $\text{round}_{e,s}$ to vector inputs by rounding coordinate-wisely.

Our notion of floating-point number simplifies the IEEE 754 Standard for Floating-point Arithmetic [4] by removing ∞ and $-\infty$. When overflow happens, we always round the output to ∞, and when underflow happens to $-\infty$. For unary functions like $\exp(\cdot)$ and binary functions including addition, subtraction, multiplication, and division, we simply define their rounded version by rounding their outputs. Whenever division by 0 happens, we define the output as 0.[1]

Next, we define finite-precision summation over more than two numbers by decomposing it as a chain of rounded binary addition in a fixed order.

Definition 2.2.3 (Summation with Iterative Rounding)**.** For any $s, n \in \mathbb{N}^+$ and vector $x \in \mathbb{R}^n$, we define summation with iterative rounding to e bit exponent and $2s$-bit precision as $\mathsf{sum}_{e,s} : \cup_{n\in\mathbb{N}^+}(\mathbb{F}_{e,s})^n \to \mathbb{F}_{e,s}$, where for any $n \in \mathbb{N}^+$ and $x \in \mathbb{R}^n$,

$$\mathsf{sum}_{e,s}(x) \triangleq \Big[\Big[\Big[[x_1 + x_2]_{e,s} + x_3\Big]_{e,s} + \cdots x_{n-1}\Big]_{e,s} + x_n\Big]_{e,s}.$$

We further define the following operations:

- Finite-precision inner product: $\langle x, y\rangle_{e,s} \triangleq \mathsf{sum}_{e,s}(x \odot y)$;
- Finite-precision matrix product: $(A \times_{e,s} B)_{i,j} \triangleq \left\langle (A_{i,:})^\top, B_{:,j}\right\rangle_{e,s}$;
- Finite-precision softmax: $\text{softmax}_{e,s}(x) \triangleq \Big[[\exp(x)]_{e,s} / \mathsf{sum}_{e,s}([\exp(x)]_{e,s})\Big]_{e,s}$.

From the saturation and the iterative rounding presented by [5], some properties appear that are important to remark:

- $exp(-\infty) = 0$
- $exp(\infty) = \infty$
- $-\infty \cdot \infty = -\infty$
- $\infty \cdot \infty = \infty$

[1] Notice that the only division is in the softmax. If it's the one in the attention mechanism, it's reasonable to give an attention score of 0 to every vector. If it's the one in the output layer it means that all symbols have an extremely small probability, therefore is reasonable to assign the same probability to all of them.

- $\forall x \in \mathbb{F}_{e,s}$ it holds that $x + 0 = x$
- $\forall x \in \mathbb{F}_{e,s}$ it holds that $x - x = 0$
- $\forall x \in \mathbb{F}_{e,s}$ it holds that $x + \infty + \infty + -\infty = 0$

2.3 Complexity theory background

In this section, we define the standard notions of complexity theory used throughout this thesis.

Definition 2.3.1 (Language)**.** In this work we consider the following notion of problems: given a sequence of input tokens, output a token as the answer. Mathematically, given a vocabulary Σ, we call a mapping $\mathcal{L} : \cup_{k \in \mathbb{N}^+} \Sigma^k \to \Sigma$ a problem. If the correct answer is always q_{accept} or q_{reject}, we call $\mathcal{L}$ a decision problem. In circuit complexity, such $\mathcal{L}$ is also called a **language**.

2.3.1 Boolean circuits

The Boolean circuits are a model of computation generalizing Boolean formulas. It is a simplified model of the silicon chips used to make modern computers. It is a natural model for nonuniform computation, as is the case for transformers; thus, most of our results on the expressiveness of transformers are stated in the language of circuit theory.

A Boolean circuit is a procedural diagram showing how to derive an output from a binary input string by applying a sequence of basic Boolean operations OR ($\vee$), AND ($\wedge$), and NOT ($\neg$) on the input bits.

Though the standard definition of circuit complexity only deals with binary strings, given any finite vocabulary Σ, we can always replace each token in Σ by its binary representation, and the length of the input only blows up by a constant factor. Therefore, we can extend existing complexity classes to arbitrary finite vocabulary naturally.

Definition 2.3.2 (Boolean circuits)**.** For every $n \in \mathbb{N}$, an n-input, single-output Boolean circuit is a directed acyclic graph with n sources (vertices with no incoming edges) and one sink (vertex with no outgoing edges). All nonsource vertices are called gates and are labeled with one of $\wedge$, $\vee$ or $\neg$ (i.e., the logical operations OR, AND, and NOT). The vertices labeled with $\vee$ and $\wedge$ have fan-in (i.e., number of incoming edges) equal to 2 and the vertices labeled with $\neg$ have fan-in 1. The size of C, denoted by $|C|$, is the number of vertices in it.

If C is a Boolean circuit, and $x \in \{0,1\}^n$ is some input, then the output of C on x, denoted by $C(x)$, is defined in the natural way. More formally, for every vertex v of C, we give it a value $val(v)$ as follows: If v is the i-th input vertex then $val(v) = x_i$ and otherwise $val(v)$ is defined recursively by applying v's logical operation on the values of the vertices connected to v. The output $C(x)$ is the value of the output vertex.

Definition 2.3.3 (Circuit families and language recognition)**.** Let $T : \mathbb{N} \to \mathbb{N}$ be a function. A $T(n)$-size circuit family is a sequence $\{C_n\}_{n \in \mathbb{N}}$ of Boolean circuits, where C_n has n inputs and a single output, and its size $|C_n| \leq T(n)$ for every n. We say that a language $\mathcal{L}$ is in $\mathsf{SIZE}(T(n))$ if there exists a $T(n)$-size circuit family $\{C_n\}_{n \in \mathbb{N}}$ such that for every $x \in \{0,1\}^n$, $x \in \mathcal{L} \iff C_n(x) = 1$.

Interesting complexity classes arise when we consider "small" circuits, such as the class of languages recognized by all polynomial-size families.

Definition 2.3.4 (P/Poly)**.** P/Poly is the class of languages that are decidable by polynomial-sized circuit families. That is, $\mathsf{P/Poly} = \cup_c \mathsf{SIZE}(n^c)$.

The class P/Poly fits rather awkwardly in the complexity world since it contains undecidable languages such as the language Unary halt[2]. The root of the problem is that for a language $\mathcal{L}$ to be in P/Poly, it suffices that a circuit family for $\mathcal{L}$ exists even if we have no way of actually constructing the circuits. Thus, it may be fruitful to try to restrict attention to circuits that can actually be built, say using a fairly efficient Turing machine.

Definition 2.3.5 (P-uniform circuit families)**.** A circuit family $\{C_n\}_{n\in\mathbb{N}}$ is P-uniform if there is a polynomial-time Turing machine that on input 1^n outputs the description of the circuit C_n.

Boolean circuits are a good model to represent parallel computation. The **depth** of a circuit is the length of the longest directed path from an input node to the output node.

Definition 2.3.6 (NC)**.** For every d, a language $\mathcal{L}$ is in NC^d if $\mathcal{L}$ can be decided by a family of circuits $\{C_n\}_{n\in\mathbb{N}}$ where C_n has $\mathsf{poly}(n)$ size and depth $O(\log^d(n))$. The class NC is $\cup_{i\geq 1}\mathsf{NC}^i$.

When studying the depth of a circuit, the fan-in of the gates becomes of interest. A polynomial fan-in can be simulated with a logarithmic blow-up in depth. This motivates the study of circuits with unbounded fan-in.

Definition 2.3.7 (AC)**.** The class AC^i is defined similarly to NC^i except gates are allowed to have unbounded fan-in (i.e., the OR and AND gates can be applied to more than two bits). The class AC is $\cup_{i\geq 0}\mathsf{AC}^i$.

AC^0 circuits allow for an interesting number of operations. For example, addition and multiplication of numbers of fixed precision can be done with AC^0 circuits. Also, the equality of two strings of bits can be done in AC^0. From the ability to compute equality it emerges that any function f with $O(\log(n))$ bits as input can be computed in AC^0. The circuit computing f enumerates the result for each of the inputs and outputs the one that correspond to the branch equal to the input. Let k be the number of bits of the input of f, then f can be represented as the following formula: $\wedge_{i=0}^{2^k} f(i) \wedge i = x$. Observe that the formula represents a circuit of constant depth. Also note that the formula has size $O(2^k)$; then, if $k \in O(\log(n))$ the circuit has polynomial size; and, therefore, it is in AC^0.

One of the main results of circuit complexity is that the MAJORITY[3] problem cannot be computed by AC^0 circuits.

Theorem 2.3.1. $\mathsf{MAJORITY} \notin \mathsf{AC}^0$

This theorem motivates the study of threshold circuits, circuits allowed to have MAJORITY gates.

RR: cita a algún lugar donde prueben eso o es lo suficientemente sabido?

[2] UHALT $= \{1^n : n$'s binary expansion encodes a pair $\langle M, x\rangle$ such that M halts on input $x\}$

[3] $\mathsf{MAJORITY} = \left\{x : \frac{\sum_{i=1}^{|x|} x_i}{|x|} > \frac{1}{2}\right\}$, i.e., the words with more 1s than 0s

Definition 2.3.8 (TC)**.** The class TC^i is defined similarly to AC^i except the circuits can have gates OR, AND, NOT and MAJORITY of unbounded fan-in. The class TC is $\cup_{i\geq 0}\mathsf{TC}^i$.

Following Theorem 2.3.1, a family of variations of MAJORITY began to be studied.

Definition 2.3.9 (MAJORITY with promise)**.** For every $e \in (1/2, 1]$ we define the problem Majority with promise (MAJ_e) as MAJORITY but restricted to words where there are at least e 0s or e 1s.

An interesting result from descriptive complexity is that for all e, there is a circuit in AC^0 recognizing MAJ_e.

RR: agregar cita

Randomized Boolean circuits

In this thesis we also work with randomized Boolean circuits. These are Boolean circuits, but they are allowed to have random gates. These gates have fan-in 0, and their value is a uniformly distributed bit. This value is fixed once the evaluation of the circuit starts. Then, the evaluation of a randomized circuit C with input x can be thought as evaluating a Boolean circuit C' with input xr, where r are the values of the random gates.

Definition 2.3.10 (Randomized Boolean circuits)**.** A randomized Boolean circuit is a directed acyclic graph with $n + r$ sources and one sink. All nonsource vertices are labeled as $\wedge$, $\vee$ or $\neg$ (i.e., the logical operations OR, AND, and NOT). The vertices labeled with $\vee$ and $\wedge$ have fan-in (i.e., number of incoming edges) equal to 2 and the vertices labeled with $\neg$ have fan-in 1. The size of C, denoted by $|C|$, is the number of vertices in it. The source vertices are either labeled as input or random.

If C is a Boolean circuit, and $x \in \{0,1\}^n$ is some input, then the output of C on x, denoted by $C(x)$, is defined as random variable in the following way. For every vertex v of C, we give it a value $val(v)$ as follows: If v is the i-th input vertex, then $val(v) = x_i$; if v is labeled as random, then $val(v)$ is a uniform random Boolean variable; and otherwise $val(v)$ is defined recursively by applying v's logical operation on the values of the vertices connected to v. The output $C(x)$ is the value of the output vertex.

In general, we say that a family of random circuits recognizes a language $\mathcal{L}$ when $\Pr[C_{|x|}(x) = \mathcal{L}(x)]$ is high.

Definition 2.3.11 (Randomized circuit families and language recognition)**.** Let $T : \mathbb{N} \to \mathbb{N}$ be a function. A $T(n)$-size randomized circuit family is a sequence $\{C_n\}_{n\in\mathbb{N}}$ of randomized Boolean circuits, where C_n has n inputs and a single output, and its size $|C_n| \leq T(n)$ for every n. We say that a language $\mathcal{L}$ is in $\mathsf{BPSIZE}(T(n))[f(n)]$ if there exists a $T(n)$-size randomized circuit family $\{C_n\}_{n\in\mathbb{N}}$ with $O(f(n))$ random gates such that for every $x \in \{0,1\}^n$, $\Pr[C_n(x) = 1] \geq 2/3 \iff \mathcal{L}(x)$.

Since every random Boolean circuit can be interpreted as Boolean circuit with the random bits padded in the input, we can also define recognition in the following way:

Definition 2.3.12. We say that a language $\mathcal{L}$ is in $\mathsf{BPSIZE}(T(n))[f(n)]$ if there exists a $T(n)$-size circuit family $\{C'_n\}_{n\in\mathbb{N}}$ such that for every $x \in \{0,1\}^n$, $\Pr_{r\in\{0,1\}^{f(n)}}[C'_n(xr) = 1] \geq 2/3 \iff \mathcal{L}(x)$.

In general, we drop the bound for the number of random bits when it is not of interest.

Definition 2.3.13. $\mathsf{BPSIZE}(T(n)) = \mathsf{BPSIZE}(T(n))[T(n)]$.

Definition 2.3.14. Similar to $\mathsf{P/Poly}$, we define $\mathsf{BPP/Poly}$ as the class of languages that are decidable by polynomial-sized random circuit families. That is, $\mathsf{BPP/Poly} = \cup_c \mathsf{BPSIZE}(n^c)$.

We extend the definition of NC, NC^i, AC, AC^i, TC, TC^i uniform and not uniform to randomized Boolean circuits in the natural way. We denote by $\mathcal{C}[f(n)]$ the class $\mathcal{C}$ restricted to $O(f(n))$ random gates.

Observe that a Boolean circuit is a random Boolean circuit without random gates.

Theorem 2.3.2. *Every circuit class $\mathcal{C}$ is contained in its corresponding randomized circuit class. For example* $\mathsf{AC}^0 \subseteq \mathsf{BPAC}^0$.

The other inclusion is not known in general. In the case of AC^0 and BPAC^0, restricting BPAC^0 to only $O(f(n))$ random bits is enough to be make it fall inside AC^0. This is because in AC^0 we can enumerate all possible values for the random bits. Then, we compute the output of the random Boolean circuit for each case. There are $O(n)$ possible random values. We can run the circuit for each of them in parallel leading only to a linear blow-up in width but constant in depth. Then, by definition of the random circuit, at least $2/3$ of the cases have the same output. Therefore, we can output $\mathrm{MAJ}_{2/3}$.

Theorem 2.3.3. $\mathsf{BPAC}^0[f(n)] \subseteq \mathsf{AC}^0$

2.3.2 Boolean circuits and Turing machines

Most of the work in complexity theory is done over Turing machines. In this section, we present some known relations between circuits and Turing machines.

Definition 2.3.15 (DTIME)**.** Let $T : \mathbb{N} \to \mathbb{N}$ be some function. A language $\mathcal{L}$ is in $\mathsf{DTIME}(T(n))$ iff there is a Turing machine that runs in time $c \cdot T(n)$ for some constant $c > 0$ and decides $\mathcal{L}$.

Definition 2.3.16. $\mathsf{P} = \cup_{c \geq 0} \mathsf{DTIME}(n^c)$.

A probabilistic Turing machine is a Turing machine with access to a random coin.

Definition 2.3.17 (BPTIME)**.** Let $T : \mathbb{N} \to \mathbb{N}$ be some function. A language $\mathcal{L}$ is in $\mathsf{BPTIME}(T(n))$ iff there is a probabilistic Turing machine M that runs in time $c \cdot T(n)$ for some constant $c > 0$ and for all $x \in \{0,1\}^n$, $x \in \mathcal{L} \iff \Pr[M(x) = 1] \geq 2/3$

Definition 2.3.18. $\mathsf{BPP} = \cup_{c \geq 0} \mathsf{BPTIME}(n^c)$.

When restricting $\mathsf{P/Poly}$ and $\mathsf{BPP/Poly}$ to only their uniform families they coincide with P and BPP. To simulate a circuit by a Turing machine, one first computes the circuit description (this can be done since we are only analyzing uniform families) and then evaluates it, both in polynomial time. For the simulation of Turing machines by circuits, the construction of Cook-Levin works.

Theorem 2.3.4. $\mathsf{Uniform} \cap \mathsf{P/Poly} = \mathsf{P}$

Theorem 2.3.5. $\mathsf{Uniform} \cap \mathsf{BPP/Poly} = \mathsf{BPP}$

An important result relating machines with and without randomness is Adleman's theorem. Adleman proved that the non-uniformity of $\mathsf{P/Poly}$ gives enough power to simulate the randomness of BPP.

RR: agregar cita a Adleman

Theorem 2.3.6 (Adleman's theorem)**.** $\mathsf{BPP} \subseteq \mathsf{P/Poly}$

Lista de cosas que voy referenciando:

- $\mathsf{MAJORITY}_p r$ con $pr > 1/2$ se puede hacer en ac0 (https://www.ccs.neu.edu/home/viola/papers/BPvsl

2.4 Transformer Complexity classes

Maybe rethink this title?

As discussed in the first chapter, there are different resources to bound in the transformers to limit their expressive power. In this thesis, we analyze the effect of bounding the number of steps in the chain of thought that the transformers are allowed to make before answering. With this in mind, we define the following complexity classes.

Given two functions $T(n)$ and $d(n)$ we define the complexity class $\mathsf{CoT}[T(n), d(n)]$. Informally, a language $\mathcal{L}$ is in $\mathsf{CoT}[T(n), d(n)]$ if and only if there is a deterministic family of transformers with constant depth, embedding size $d(n)$ and budget $T(n)$ that recognizes it.

Definition 2.4.1 ($\mathsf{CoT}[T(n), d(n)]$)**.** A language $\mathcal{L}$ is in $\mathsf{CoT}[T(n), d(n)]$ if and only if there is an integer L and two functions $T'(n) = O(T(n)), d'(n) = O(d(n))$, such that for every positive integer n, there is an L-layer deterministic transformer, denoted by TF_n with embedding size $d'(n)$, that outputs $\mathcal{L}(\alpha)$ given any input α in Σ^n, using $T'(n)$ steps of chain of thought.

In the same way, we define the classes of languages recognized by probabilistic transformers. Given two functions $T(n)$ and $d(n)$ we define the complexity class $\mathsf{RCoT}[T(n), d(n)]$. Informally, a language $\mathcal{L}$ is in $\mathsf{RCoT}[T(n), d(n)]$ if and only if there is a probabilistic family of transformers with constant depth, embedding size $d(n)$ and budget $T(n)$ that recognizes it.

Definition 2.4.2 ($\mathsf{RCoT}[T(n), d(n)]$)**.** A language $\mathcal{L}$ is in $\mathsf{RCoT}[T(n), d(n)]$ if and only if there is an integer L and two functions $T'(n) = O(T(n)), d'(n) = O(d(n))$, such that for every positive integer n, there is an L-layer probabilistic transformer, denoted by TF_n with embedding size $d'(n)$, that outputs $\mathcal{L}(\alpha)$ given any input α in Σ^n, using $T'(n)$ steps in the chain of thought with probability higher than $2/3$.

2.5 Known results on transformer's power

The main objective of this thesis is to expand on the state-of-the-art bounds for transformer expressiveness. We adapt [5]'s results from deterministic transformers to probabilistic transformers. Their main results imply that $\mathsf{CoT}[poly(n), log(n)] = \mathsf{P/Poly}$.

2.5.1 Upper bound

[5] expands on the work of [10]. [10] have shown that one step of a constant-depth, polynomial-embedding-dimension and log-precision deterministic transformer can be simulated in a small parallel time, i.e., using TC^0 circuits. This result is built on the fact that multiplication and division of n-bit binary numbers are in TC^0 [3], as well as the iterated addition over n different n-bit binary integers.

However, such TC^0 expressiveness upper bounds may be unrealistic for transformers operating with floating-point numbers. In [10], it is implicitly assumed that, when adding more than one floating-point number, the algorithm first computes the exact answer without rounding, using arbitrarily higher precision, and performs rounding only at the end. However, in practice, rounding occurs after each addition of two numbers, and it remains open whether such TC^0 upper bounds still hold. Immediate rounding makes iterated addition over floating-point numbers no longer associative [2]. The associativity of integer addition is essential for the result that iterated addition over n different n-bit binary integers is in TC^0.

The abstraction for transformers presented by [5] uses the numeric representation presented in Section 2.2. Their model rounds after every numerical operation and represents numbers with a constant number of bits for the significand and the exponent.

[5] prove that $\mathsf{sum}_{e,s} : \mathbb{F}_{e,s}^n \to \mathbb{F}_{e,s}$ has an AC^0 circuit when e, s are both constants. Then, composing circuits, they simulate one step of a deterministic, constant-depth, polynomial-embedding-dimension transformer with a circuit in AC^0 (Theorem 2.5.1).

Theorem 2.5.1 (Upper bound for deterministic transformers)**.** $\mathsf{CoT}[0, \mathsf{poly}(n)] \subseteq \mathsf{AC}^0$

Observe that the fact that a single forward pass of a transformer can be simulated by an AC^0 circuit immediately implies that a transformer with $O(\log(n))$ steps of CoT can also be simulated by AC^0 (Corollary 2.5.1). This is because a transformer with T steps of CoT can be interpreted as an OR of $|\Sigma|^T$ disjoint subcircuits, where each of them enumerates all possible sequences of T chain-of-thought tokens and outputs the value of the token in the branch where all the intermediate token values are consistent.

We say a sequence of tokens $\sigma_1, \ldots, \sigma_T$ is consistent with the input α when it satisfies $\mathsf{TF}(\alpha) = \sigma_1$, $\mathsf{TF}(\alpha\sigma_1) = \sigma_2$, $\ldots$, $\mathsf{TF}(\alpha\sigma_1 \ldots \sigma_{T-1}) = \sigma_T$. Each of the conditions can be checked by an AC^0 circuit in parallel. Then, for each sequence of tokens there is an AC^0 circuit validating its consistency. Since the transformer is deterministic, there is only one consistent sequence. The circuit outputs the value of the last token in the consistent sequence.

The enumeration of all possible sequences can be done in parallel and thus only takes constant depth. When $T = O(\log(n))$, this only leads to a $\mathsf{poly}(n)$ blow-up in circuit size. Therefore, it follows that $\mathsf{CoT}[\log(n), \mathsf{poly}(n)] \subseteq \mathsf{AC}^0$.

Corollary 2.5.1. $\mathsf{CoT}[\log(n), \mathsf{poly}(n)] \subseteq \mathsf{AC}^0$

Since each pass of the transformer can be simulated by a circuit in AC^0, a transformer performing a polynomial number of CoT steps $T(n)$ can be simulated by a circuit of depth $T(n)$ (and polynomial size), by stacking the circuit for one pass. Therefore, a transformer performing a polynomial number of steps can be simulated by a circuit in $\mathsf{P/Poly}$.

Corollary 2.5.2. $\mathsf{CoT}[\mathsf{poly}(n), \mathsf{poly}(n)] \subseteq \mathsf{P/Poly}$

2.5.2 Lower bound

[5] also prove a lower bound on the expressiveness of constant-depth, constant-precision deterministic transformers using CoT with $O(\log(n))$ embedding size (Theorem 2.5.2).

Theorem 2.5.2 (Lower bound for deterministic transformers)**.** *For any polynomial function* $T : \mathbb{N}^+ \to \mathbb{N}^+$, $\mathsf{SIZE}(T(n)) \subseteq \mathsf{CoT}[T(n), \log(n)]$. *In particular,* $\mathsf{P/Poly} \subseteq \mathsf{CoT}[\mathsf{poly}(n), \log(n)]$.

[5] construct, for each circuit, a transformer such that at each CoT step the output of one gate of the circuit is written to the tape.

Let $\{C_n\}_{n\in\mathbb{N}}$ be a family of circuits of polynomial size. [5] construct a family $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ such that each TF_n simulates C_n.

TF_n evaluates C_n gate by gate, processing one gate per CoT step. Let us assign an index to each gate of C_n in topological order, such that the input gates receive indices 1 through n. Then, in the i-th step of CoT, TF_n prints the value of the $(n+i)$-th gate, i.e., $\mathsf{TF}_n^i(\alpha)$ equals the value of the $(n+i)$-th gate when the input is α. Thus, at the end of the computation, the i-th position of the tape contains the value of the i-th gate.

Their construction encodes the information of each gate of the circuit into the position encoding. This information is encoded in binary, i.e., each coordinate takes the value 0 or 1. They store in the i-th vector the current gate index (i), the type (AND, OR, or NOT) of the $(i+1)$-th gate, and the indices of its two input gates. The requirement of $d(n) = \Theta(\log(n))$ embedding size comes from the fact that there are $\mathsf{poly}(n)$ gates and all of their indices must be uniquely representable in binary.

Then, at the i-th step of the computation, the attention mechanism is used to retrieve the values of the two input gates of the $(n+i)$-th gate. Next, a feed-forward network computes the value of the $(n+i)$-th gate based on its type and the values of its input gates retrieved in the previous step. Finally, in the last CoT step, TF_n outputs the value of the output gate (sink) of C_n.

3. GENERAL PROPERTIES

In this chapter we study the properties of $\mathsf{CoT}[T(n), d(n)]$ classes and the primitive operations that transformers can perform. We define the notion of normalized transformer and prove that $\mathsf{CoT}[T(n), d(n)]$ classes are closed under boolean operations.

3.1 Transformers Primitives

In this section we introduce a set of primitive operations that are used through the thesis. The operations are applied as modifications to an already existing transformer. If the operations are applied to an entire family of transformers $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ such that the language it computes is in $\mathsf{CoT}[T(n), d(n)]$, then the language of the modified family will be in the same class. This is because, although the operations add layers and coordinates to the size of the embedding, this is only a constant factor.

Most operations don't make sense on their own, but they are used in later demonstrations.

RR agregar que no es una locura tener primitivas. https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html

3.1.1 Flagging Tokens and Positions

For complex operations inside the loop of the transformer we will need to identify when a vector comes from a certain token or position. In this section we present how to mark the vectors with flags during the embedding.

For flagging a token we use the token embedding, and in the same way, for flagging a position we use the position encoding. Either way, the strategy is exactly the same. The embedding dimension is extended to allocate the flags. A new coordinate is added per flag. Therefore, using a constant number of flags only adds a constant to the embedding dimension. Thus, if the flagging is made in a family of transformers its $\mathsf{CoT}[T(n), d(n)]$ class does not change.

For example, let v_σ and v_τ be vectors of a transformer TF where v_σ comes from embedding the token σ and v_τ from the token τ (with $\sigma \neq \tau$). We can define a transformer TF' that behaves like TF but recognize when a vector comes from embedding the token σ. TF' has embedding dimension $d' = d + 1$ where d is the embedding dimension of TF.

$$v_\sigma = \begin{pmatrix} x \\ y \\ z \end{pmatrix} \to v'_\sigma = \begin{pmatrix} x \\ y \\ z \\ f_{\mathsf{on}} \end{pmatrix} \qquad v_\tau = \begin{pmatrix} i \\ j \\ k \end{pmatrix} \to v'_\tau = \begin{pmatrix} i \\ j \\ k \\ f_{\mathsf{off}} \end{pmatrix}$$

Fig. 3.1: Change in a flagged and unflagged vector

Formally, for flagging the token σ we define the token embedding function of TF' : $\Sigma \to \mathbb{F}_{e,s}^{d+1}$ as

$$(\theta'_{TE}(\tau))_j = \begin{cases} (\theta_{TE}(\tau))_j & \text{if } j \neq d+1 \\ f_{\mathsf{on}} & \text{if } j = d+1 \text{ and } \tau = \sigma \\ f_{\mathsf{off}} & \text{if } j = d+1 \text{ and } \tau \neq \sigma \end{cases}$$

where f_{on} is the value that marks the vector as flagged and f_{off} as unflagged. Those values can be defined chosen arbitrarily for each flag. In this case, the coordinate inserted for the flag is the last one, but it can be added anywhere.

The other components of TF' ignore $d+1$ the coordinate. The position encoding of TF' is equal to the position encoding of TF, except it puts a 0 in the $d+1$ coordinate (not present in TF) independently of the position. The matrices of the ATTN, FF and OUTPUT layers of TF' are the ones of TF extended with 0 in the $d+1$ rows and columns to match the change in embedding dimension.

TF' behaves exactly as TF except it has one more coordinate in the embedding. Said coordinate represents the flag and takes only the values v_{on} or v_{off} in every vector.

This primitive works also as a mechanism to extend the embedding dimension without changing the behavior. If v_{on} and v_{off} are defined as 0, the resulting transformer is the same as the initial with an extra coordinate that is always 0.

3.1.2 Preserve and ignore coordinates through ATTN and FF

When defining new transformers over existing ones we will need to extend the embedding dimension to store new information. In order to preserve the behavior of the original transformers across the attention and feed-forward layers, we need to construct a mechanism that ignores the extra coordinates. For example, this mechanism is used for preserving flags during ATTN and FF layers that don't use them.

Let TF be a transformer of embedding dimension d and assume we are constructing a transformer TF' of embedding dimension $d' = d+1$. Let W_Q, W_K, W_V, W_O be the matrices of an ATTN layer of TF and assume we want to perform the same layer in TF' but simulating that the ith coordinate does not exist. Notice that the embedding dimension of TF' has one more coordinate than TF's. If we define the matrices of the layer W'_Q, W'_K, W'_V, W'_O as W_Q, W_K, W_V, W_O by inserting a row and column of zeros at the ith position, we obtain the desired ATTN layer.

Note that because the ith row of W'_O is zero, the output vectors of the ATTN layer are 0 in the ith coordinate. This is the point that makes it possible to not just ignore the value of the ith coordinate, but to not affect it.

For preserving the coordinate but not ignore it, it is enough to have zeros in only its corresponding row of W'_O. This construction works for more than one coordinate. To extend it to a set of coordinates, it suffices to set more rows and columns to 0.

The same applies in the feed forward layer, making the ith column and row of the matrices and vectors of the layer zero, makes it ignore the value of the ith coordinate. Furthermore, the result of the FF layer will have a zero in the ith coordinate, thus preserving its value.

SC diagrama?

RR how? pongo las matrices? Siento que a veces queda muy cargado. Quizás ahora se entiende mejor?

3.1.3 Add vector and constant functions

In this section we develop a mechanism for adding a fixed vector to a previously flagged one. This technique works even when there is more than one flagged vector.

Assume there is a flag with values $f_{\mathsf{on}} = 1$ and $f_{\mathsf{off}} = 0$. For clarity, suppose the flag is in the last coordinate. Let v be the vector we want to add. We define a mechanism that replaces every h h such that $h_d = f_{\mathsf{on}}$ with $h + v$.

The operation is performed with only one FF layer. We construct a layer such that $FF(h) = \mathbb{I}(h_d = f_{\mathsf{on}}) \cdot v$. Said layer is achieved with the following parameters.

$$W_1 = id \qquad W_2 = \begin{pmatrix} 0 & \dots & 0 & v_1 \\ \vdots & \ddots & \vdots & \vdots \\ 0 & \dots & 0 & v_d \end{pmatrix} \qquad b_1 = b_2 = \begin{pmatrix} 0 \\ \vdots \\ 0 \end{pmatrix}$$

Notice that if the coordinate for the flag had been the kth instead of the last one, then v would have appeared in the kth column of W_2 instead of the last.

This technique can be used to apply a constant function to the flagged vectors. This is achieved by exploiting the finite representation system. We can saturate the vector to the ∞ and then make it 0 to finally add our objective. This holds since for all $x, c \in \mathbb{F}_{e,s}$ it happens that $x+\infty+\infty+-\infty+c = c$. Because as mentioned in Section 2.2, $x+\infty+\infty = \infty$ and $\infty + -\infty = 0$.

The constant function $f(h) = v$ is implemented by adding four vectors that perform this trick. First, it is necessary to add the vector consisting entirely of ∞ twice; then, a vector consisting entirely of $-\infty$; and finally, v; as shown in equation 3.1.

$$h \leftarrow h + \begin{pmatrix} \infty \\ \vdots \\ \infty \end{pmatrix} + \begin{pmatrix} \infty \\ \vdots \\ \infty \end{pmatrix} + \begin{pmatrix} -\infty \\ \vdots \\ -\infty \end{pmatrix} + v \tag{3.1}$$

3.1.4 Make a vector ignore others in an ATTN layer

We show how to make a vector ignore others in the ATTN layers, a technique that will prove useful in later constructions. A vector h_i ignores vector h_j when the attention score is 0, which can be achieved by making $\langle q_i, k_j \rangle = -\infty$.

We extend the embedding dimension with three more coordinates for preserving the attention score of the other vectors previous to taking softmax. We construct TF' over TF such that the lth vector ignores a set of vectors M in an ATTN layer; specifically, we require said layer to satisfy:

$$\langle q'_i, k'_j \rangle = \begin{cases} -\infty & \text{if } i = l \wedge j \in M \\ \langle q_i, k_j \rangle & \text{if } i \neq l \vee j \notin M \end{cases}$$

This can be achieved by saturating the finite representation since $\langle q'_l, k'_i \rangle = \sum_{x=1}^{d+3} (q'_l)_x (k'_i)_x$. If the last two terms of the summation are $-\infty$, then because of the iterative rounding, $\langle q'_l, k'_i \rangle = -\infty$. Since we do not want to change the attention scores of vectors not in M, when $i \notin M$ the last three terms should be 0.

We force $(k'_i)_{d+2}$ and $(k'_i)_{d+3}$ to be $-\infty$ when $i \in M$, and $(q'_i)_{d+2}$ and $(q'_i)_{d+3}$ to be 1 when $i = l$ and 0 otherwise. We also make $(k'_i)_{d+1}$ and $(q'_i)_{d+1}$ to be 0 for all i so the third to last term of the inner product is always 0. With these values, the only attention scores modified are the ones of the lth vector since every other will have 0 in the last three coordinates of the query thus making the last three terms of the inner product 0 and therefore not affecting the score.

This can be achieved with the following flags:

$$(h_i)_{d+1} = \begin{cases} 1 & \text{if } i = l \\ 0 & \text{otherwise} \end{cases} \qquad (h_i)_{d+2} = (h_i)_{d+3} = \begin{cases} 1 & \text{if } i \in M \\ 0 & \text{otherwise} \end{cases}$$

Now the query and key matrices are extended as:

$$W'_Q = \begin{pmatrix} W_Q & 0^{\mathbb{F}_{e,s}^{d\times 1}} & 0^{\mathbb{F}_{e,s}^{d\times 1}} & 0^{\mathbb{F}_{e,s}^{d\times 1}} \\ 0^{\mathbb{F}_{e,s}^{d\times 1}} & 0^{\mathbb{F}_{e,s}^{1\times 1}} & 0^{\mathbb{F}_{e,s}^{1\times 1}} & 0^{\mathbb{F}_{e,s}^{1\times 1}} \\ 0^{\mathbb{F}_{e,s}^{d\times 1}} & 1^{\mathbb{F}_{e,s}^{1\times 1}} & 0^{\mathbb{F}_{e,s}^{1\times 1}} & 0^{\mathbb{F}_{e,s}^{1\times 1}} \\ 0^{\mathbb{F}_{e,s}^{d\times 1}} & 1^{\mathbb{F}_{e,s}^{1\times 1}} & 0^{\mathbb{F}_{e,s}^{1\times 1}} & 0^{\mathbb{F}_{e,s}^{1\times 1}} \end{pmatrix} \quad W'_K = \begin{pmatrix} W_K & 0^{\mathbb{F}_{e,s}^{d\times 1}} & 0^{\mathbb{F}_{e,s}^{d\times 1}} & 0^{\mathbb{F}_{e,s}^{d\times 1}} \\ 0^{\mathbb{F}_{e,s}^{d\times 1}} & 0^{\mathbb{F}_{e,s}^{1\times 1}} & 0^{\mathbb{F}_{e,s}^{1\times 1}} & 0^{\mathbb{F}_{e,s}^{1\times 1}} \\ 0^{\mathbb{F}_{e,s}^{d\times 1}} & 0^{\mathbb{F}_{e,s}^{1\times 1}} & -\infty^{\mathbb{F}_{e,s}^{1\times 1}} & 0^{\mathbb{F}_{e,s}^{1\times 1}} \\ 0^{\mathbb{F}_{e,s}^{d\times 1}} & 0^{\mathbb{F}_{e,s}^{1\times 1}} & 0^{\mathbb{F}_{e,s}^{1\times 1}} & -\infty^{\mathbb{F}_{e,s}^{1\times 1}} \end{pmatrix}$$

Then,

$$\begin{aligned} \langle q'_i, k'_j \rangle &= \sum_{x=1}^{d+3} (q'_l)_x (k'_i)_x = \langle q_i, k_j \rangle + (q'_l)_{d+1}(k'_i)_{d+1} + (q'_l)_{d+2}(k'_i)_{d+2} + (q'_l)_{d+3}(k'_i)_{d+3} \\ &= \langle q_i, k_j \rangle + 0 + (q'_l)_{d+2}(k'_i)_{d+2} + (q'_l)_{d+3}(k'_i)_{d+3} \\ &= \langle q_i, k_j \rangle + (q'_l)_{d+2}(k'_i)_{d+2} + (q'_l)_{d+3}(k'_i)_{d+3} \\ &= \begin{cases} \langle q_i, k_j \rangle + 1 \cdot -\infty + 1 \cdot -\infty & \text{if } i = l \wedge j \in M \\ \langle q_i, k_j \rangle + 1 \cdot 0 + 1 \cdot 0 & \text{if } i = l \wedge j \notin M \\ \langle q_i, k_j \rangle + 0 \cdot -\infty + 0 \cdot -\infty & \text{if } i \neq l \wedge j \in M \\ \langle q_i, k_j \rangle + 0 \cdot 0 + 0 \cdot 0 & \text{if } i \neq l \wedge j \notin M \end{cases} \\ &= \begin{cases} \langle q_i, k_j \rangle + -\infty + -\infty & \text{if } i = l \wedge j \in M \\ \langle q_i, k_j \rangle + 0 + 0 & \text{if } i = l \wedge j \notin M \\ \langle q_i, k_j \rangle + 0 + 0 & \text{if } i \neq l \wedge j \in M \\ \langle q_i, k_j \rangle + 0 + 0 & \text{if } i \neq l \wedge j \notin M \end{cases} \\ &= \begin{cases} -\infty & \text{if } i = l \wedge j \in M \\ \langle q_i, k_j \rangle & \text{otherwise} \end{cases} \end{aligned}$$

Namely, $\langle q'_i, k'_j \rangle = -\infty$ if $i = l$ and $j \in M$, but $\langle q'_i, k'_j \rangle = \langle q_i, k_j \rangle$ otherwise.

3.1.5 Linear transformations

Although it may seem that transformers are a model that should be able to perform linear transformations in a native way, it is not the case. Applying a linear transformation to a vector is not straightforward, since in every step of ATTN and FF the result of the layer is added to the original vector instead of replacing it. An intuitive idea would be to apply the transformation $M - I$ when we want to transform the vector x to Mx, since $x + (M - I)x = x + Mx - Ix = Mx$. This does not hold in our model since the addition

and multiplication are not commutative in $\mathbb{F}_{e,s}$. For avoiding the representation error, we will extend the embedding dimension to get more memory per vector such that we can store in new coordinates the value of Mx and then replace it in the original ones.

Let f denote the index of the flagged vector. We show a mechanism to replace it with the result of applying the linear transformation M to it. We perform the operation with two consecutive ATTN layers, in which h_f only attends to itself. Therefore, after these two layers h_f will be replaced with Mh_f.

We need to double the embedding size from d to $2d$ (plus some flags) since we need to store the result of the transformation in some coordinates (the added $d+1$ to $2d$) as a middle step. In the first ATTN layer, we compute the transformation saving its result in the coordinates $d+1$ to $2d$. Then, in the second layer we move the result to the right place.

$$\widetilde{h_f} = \begin{pmatrix} (h_f)_1 \\ \vdots \\ (h_f)_d \\ 0 \\ \vdots \\ 0 \end{pmatrix} \xrightarrow{\text{first ATTN layer}} \widetilde{h_f} = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ (Mh_f)_1 \\ \vdots \\ (Mh_f)_d \end{pmatrix} \xrightarrow{\text{second ATTN layer}} \widetilde{h_f} = \begin{pmatrix} (Mh_f)_1 \\ \vdots \\ (Mh_f)_d \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

We assume that the $d+1$ to $2d$ coordinates (used as memory) are set to 0 by the layers preceding this mechanism This can be achieved maintaining them as 0 since the beginning or transformed to 0 with the primitive described in Section 3.1.3.

Let us construct each of the ATTN layers. As said before, in both layers we will need for $\widetilde{h_f}$ to only attend to itself. Therefore, we define W_Q and W_K using the strategy of Section 3.1.4.

Recall from the definition of the ATTN mechanism (1) that the output of the ATTN layer is $h'_i = W_O \sum_{j=1}^{a} (s_i)_j v_j$. By this point, h'_f is looking like

$$h'_f = W_O \sum_{j=1}^{a} (s_f)_j v_j = W_O v_f = W_O (W_V \widetilde{h_f})$$

- For the first layer, defining W_O and W_V as follows

  $$W_V = id \qquad\qquad W_O = \begin{pmatrix} -id^{\mathbb{R}^{d\times d}} & 0^{\mathbb{R}^{d\times d}} \\ M & 0^{\mathbb{R}^{d\times d}} \end{pmatrix}$$

  makes us get $h'_f = (-(h_f)_1, \ldots, -(h_f)_d, (Mh_f)_1, \ldots, (Mh_f)_d)$, which added to $\widetilde{h_f}$ gives us what we were looking for after the first layer.

- For the second ATTN layer, we use the same mechanism for making $\widetilde{h_f}$ only attend to itself. And defining W_V and W_O as follows

  $$W_V = id \qquad\qquad W_O = \begin{pmatrix} 0^{\mathbb{R}^{d\times d}} & id^{\mathbb{R}^{d\times d}} \\ 0^{\mathbb{R}^{d\times d}} & -id^{\mathbb{R}^{d\times d}} \end{pmatrix}$$

  we get $h'_f = ((Mh_f)_1, \ldots, -(Mh_f)_d, -(Mh_f)_1, \ldots, -(Mh_f)_d)$, which added to $\widetilde{h_f}$ gives us what we were looking for after the second layer.

Note that, after these two layers, the values of h_i for all $i \neq f$ have changed in ways that have not been studied in this paper, but this is not relevant to the way we use the primitive.

3.1.6 Max coordinate

We show how to take the maximum of a set of coordinates. This operation will be needed for later constructions.

We give a construction for taking the maximum of two coordinates. This construction can be easily extended to more coordinates. The operation works properly when the set of coordinates has only one maximum. All the coordinates that are not the maximum are replaced by 0.

$$\begin{pmatrix} a \\ b \\ \vdots \end{pmatrix} \to \begin{cases} \begin{pmatrix} a \\ 0 \\ \vdots \end{pmatrix} & \text{if } a > b \\ \\ \begin{pmatrix} 0 \\ b \\ \vdots \end{pmatrix} & \text{if } b > a \end{cases}$$

The maximum of two coordinates can be computed with the following primitives:

$$\begin{pmatrix} a \\ b \\ 0 \\ 0 \\ \vdots \end{pmatrix} \xrightarrow{\text{FF layer}} \begin{pmatrix} a \\ b \\ \text{relu}(a-b) \\ \text{relu}(b-a) \\ \vdots \end{pmatrix} \xrightarrow[\text{the coordinates with the relu}]{\text{Multiply twice by } \infty} \begin{pmatrix} a \\ b \\ \infty \cdot \mathbb{I}(a > b) \\ \infty \cdot \mathbb{I}(b > a) \\ \vdots \end{pmatrix} \longrightarrow$$

$$\xrightarrow[\text{coordinates with the comparison}]{\text{Divide by } \infty \text{ the}} \begin{pmatrix} a \\ b \\ \mathbb{I}(a > b) \\ \mathbb{I}(b > a) \\ \vdots \end{pmatrix} \xrightarrow[\text{operation}]{\text{Special}} \begin{pmatrix} a \cdot \mathbb{I}(a > b) \\ b \cdot \mathbb{I}(b > a) \\ 0 \\ 0 \\ \vdots \end{pmatrix} = \begin{cases} \begin{pmatrix} a \\ 0 \\ \vdots \end{pmatrix} & \text{if } a > b \\ \\ \begin{pmatrix} 0 \\ b \\ \vdots \end{pmatrix} & \text{if } b > a \end{cases}$$

For computing the maximum of a set of coordinates it is only needed to compute a comparison for each pair of the set and multiply each coordinate x by $\mathbb{I}(x > y)$ for all $y \neq x$ in the set.

Implementation of the special operation

This operation is not a linear transformation; therefore, we need to define a new technique. Observe that one of $\mathbb{I}_a$ and $\mathbb{I}_b$ equals 1 and the other equals 0. Then, $\mathbb{I}_b = 1 - \mathbb{I}_a$ and

$\mathbb{I}_a = 1 - \mathbb{I}_b$.

$$\begin{pmatrix} a \\ b \\ \mathbb{I}_a \\ \mathbb{I}_b \end{pmatrix} \xrightarrow[\text{operation}]{\text{Special}} \begin{pmatrix} a \cdot \mathbb{I}_a \\ b \cdot \mathbb{I}_b \\ 0 \\ 0 \end{pmatrix}$$

This operation is performed with three layers of FF. Naturally, after each layer a new vector is added to the original. The vectors we add are the following:

$$\begin{pmatrix} a \\ b \\ \mathbb{I}_a \\ \mathbb{I}_b \end{pmatrix} \xrightarrow[\text{operation}]{\text{Special}} \begin{pmatrix} a & + & \infty \cdot \mathbb{I}_b & + & \infty \cdot \mathbb{I}_b & + & -\infty \cdot \mathbb{I}_b \\ b & + & \infty \cdot \mathbb{I}_a & + & \infty \cdot \mathbb{I}_a & + & -\infty \cdot \mathbb{I}_a \\ \mathbb{I}_a & + & 0 & + & 0 & + & -\mathbb{I}_a \\ \mathbb{I}_b & + & 0 & + & 0 & + & -\mathbb{I}_b \end{pmatrix}$$

Note that if $\mathbb{I}_a = 1$, then $\mathbb{I}_b = 0$, so the first coordinate isn't modified and the second is changed to 0 because of the finite representation properties. In the same way, if $\mathbb{I}_a = 0$, then $\mathbb{I}_b = 1$, so the second coordinate isn't modified and the first is changed to 0.

Let us show how to instantiate the FF layers. The first two have parameters:

$$W_1 = id \qquad W_2 = \begin{pmatrix} 0 & 0 & 0 & \infty \\ 0 & 0 & \infty & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \qquad b_1 = b_2 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Notice that the output of these FF layer when the input is $(x, y, \mathbb{I}_a, \mathbb{I}_b)$ will be $(\infty \cdot \mathbb{I}_b, \infty \cdot \mathbb{I}_a, 0, 0)$.

The third FF layer has parameters:

$$W_1 = id \qquad W_2 = \begin{pmatrix} 0 & 0 & 0 & -\infty \\ 0 & 0 & -\infty & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \qquad b_1 = b_2 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

which, when the input is $(x, y, \mathbb{I}_a, \mathbb{I}_b)$, outputs $(-\infty \cdot \mathbb{I}_b, -\infty \cdot \mathbb{I}_a, -\mathbb{I}_a, -\mathbb{I}_b)$

RR intenté escribir cómo quedaban las FF instanciadas pero queda medio ilegible. No sé si vale la pena

3.1.7 Repeat a token once it's printed

We show how to make a transformer repeat a token until we decide it to make it stop. This construction is needed to argue that for every transformer TF there is a transformer TF' of the same $\mathsf{CoT}[T(n), d(n)]$ class that computes the same language and consumes all of its budget.

Assume that TF is normalized and let us define TF'. TF' is identical to TF except for the following modifications. Let σ be the token to repeat. We flag the vectors coming from σ with the token embedding. Since we are replicating a token that was printed, i.e., it is not part of the input, its corresponding vector is the last one. This is because we begin repeating it after the first time it is printed.

Since we flag the vectors coming from the token σ we can apply a constant function only to them. This function is applied in the last layer of the body of TF'. Since we assume TF was normalized, the OUTPUT matrix is just a projection. Then, the value of the constant function is what is outputted as the probability distribution (after softmax). If the constant function has value $-\infty$ in every coordinate except in the one associated with the probability of σ, where it has ∞, the distribution outputted by the distribution generator will have all the probability concentrated in σ. Thus, TF' prints σ.

Observe that this only works if the token does not appear in the input word. This case can be handled by exploiting the non-uniformity of the model. If the input word is of size n, with the PE we can flag the first n vectors. Then, we can use that flag to set to 0 the coordinate used by the TE for flagging the token to be repeated, i.e., turning of the flag. We make them 0 using a constant function as described in the Section 3.1.3.

3.1.8 Force to halt after certain number of steps of CoT

Using a strategy similar to the one in Section 3.1.7, we use the PE to flag when a certain position is reached. Then, if the transformer is normalized, modifying the last vector we decide the outputted distribution. Particularly, we can force it to concentrate the probability in q_{accept} or q_{reject}. We change the last vector with a constant function. The flag inserted by the PE is to decide if the function has to be applied or not.

Notice that this primitive, together with the one described in Section 3.1.7, proves that for every transformer TF in $\mathsf{CoT}[T(n), d(n)]$ there is a transformer TF' in the same class that always consumes all of its budget.

We extend the alphabet with two dummy versions of the decision tokens and construct TF' over TF. We make TF' work as TF, but we make it print the dummy versions of the decision tokens when TF would have printed the real ones[1]. Then, we force TF' to repeat the dummy decision tokens until it reaches the last position of the tape (uses all the budget). At that point, we force TF' to print the real decision token corresponding to the one it was repeating.

3.1.9 Copy one vector into another with ATTN

For the disjunction of transformers we will need to move values from the ith vector to the jth vector. In this section we show how to do it. This operation can only be done if $i < j$, since the attention mechanism only allows for a vector to attend to vectors to its left, i.e., h_k can attend to h_l if and only if $k \leq l$.

Let j be the number of the receiving vector and i the number of the copied, i.e., we copy from h_i to h_j. The operation is done in two steps.

First, we make 0 the coordinates of h_j that will receive the values. This is done with a constant function.

Second, we make h_j only attend to h_i in an attention layer. In said attention layer, we use $W_V = id$ and W_O such that it outputs the desired coordinates of h_i in the target coordinates. Formally, W_O has only 0 in the row k if h_j is not receiving anything from h_i in the kth coordinate. If h_j is receiving the coordinate l of h_i in the position k, then the kth row of W_O has 0 in every position except the lth.

[1] This is achieved giving the probability of each decision token to its dummy version.

For example, if we want to copy the 2nd coordinate of h_i to the 3rd coordinate of h_j, W_O would look like 3.2. Then, the vector coming out of the ATTN layer will have 0 in every coordinate, except in the 3rd, where it will be $(h_i)_2$.

$$W_O = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} \tag{3.2}$$

3.2 Normalization

For an easier handling of transformers, we define a notion of normalized transformer. This notion will specify properties of the OUTPUT matrix and of the vector consumed by this step.

Definition 3.2.1 (Normalized deterministic transformer)**.** We say a deterministic transformer is normalized if it satisfies two conditions.

- N1: The matrix of the OUTPUT has to be a projection, i.e., it is a diagonal matrix with ones on its diagonal.
- N2: The last vector after the last iteration has to only be composed of $-\infty$ in every projected coordinate except one, which has to be ∞. The non projected coordinates are free to be any value.

The motivation for this definition is to simplify the OUTPUT layer (N1) and to have more control over the probability distribution generated (N2). The exact values of the last vectors are such that after projecting and applying softmax in the OUTPUT layer we get a deterministic distribution with 100% of the probability concentrated in only one token.

For every family of transformers $\mathsf{TF}_{n n\in\mathbb{N}} \in \mathsf{CoT}[T(n), d(n)]$ computing the language $\mathcal{L}$, there is a family $\mathsf{TF}'_{n n\in\mathbb{N}}$ in the same class, computing the same language, where for all n, TF'_n is normalized. In the next section we introduce an algorithm for normalizing transformers.

3.2.1 Algorithm for normalizing

For normalizing transformers, first we make them meet N1 and then N2.

Algorithm for N1

Notice that every linear transformation $M : \mathbb{R}^d \to \mathbb{F}_{e,s}^{|\Sigma|}$ with $d \geq |\Sigma|$ can be decomposed into two steps. [2]

$$\begin{pmatrix} x_1 \\ \vdots \\ x_d \end{pmatrix} \xrightarrow{M'} \begin{pmatrix} (Mx)_1 \\ \vdots \\ (Mx)_{|\Sigma|} \\ v \end{pmatrix} \xrightarrow{projection} \begin{pmatrix} (Mx)_1 \\ \vdots \\ (Mx)_{|\Sigma|} \end{pmatrix}$$

[2] We can always add more coordinates to the embedding if $d < |\Sigma|$. Since the language is fixed, it will be a constant number of coordinates, thus not changing the class of the transformer.

where $v \in \mathbb{F}_{e,s}^{d-|\Sigma|}$ can have any values.

Let TF be a transformer and M be the matrix of the OUTPUT layer. Using the decomposition just presented, and the strategy defined in Section 3.1.5 for applying linear transformations, we can change the matrix of the OUTPUT layer to be only a projection. The linear transformation M has to be applied to the last vector, since it is the one going from the body of TF to the OUTPUT layer.

Algorithm for N2

Let TF be a transformer satisfying N1, we give an algorithm for making it also meet N2.

We use the maximum operation described in Section 3.1.6 to make all the positions of the last vector 0 except the one that was the maximum before. Then, if the maximum is positive, multiplying by ∞, then subtracting 1 from every coordinate and multiplying by ∞ again makes TF meet N2.

$$\begin{pmatrix} 0 \\ \vdots \\ 0 \\ x_{max} \\ 0 \\ \vdots \\ 0 \end{pmatrix} \xrightarrow[\infty]{\text{multiplying by}} \begin{pmatrix} 0 \\ \vdots \\ 0 \\ \infty \\ 0 \\ \vdots \\ 0 \end{pmatrix} \xrightarrow{\text{subtracting 1}} \begin{pmatrix} -1 \\ \vdots \\ -1 \\ \infty - 1 \\ -1 \\ \vdots \\ -1 \end{pmatrix} \xrightarrow[\infty]{\text{multiplying by}} \begin{pmatrix} -\infty \\ \vdots \\ -\infty \\ \infty \\ -\infty \\ \vdots \\ -\infty \end{pmatrix}$$

It could be the case that the maximum was negative, in that case the resulting vector is not what we presented. We show a mechanism for making the maximum strictly positive so we can insert it in the algorithm before subtracting 1.

At this point our vector looks like $(0, \ldots, 0, x_{max}, 0, \ldots, 0)$, we are going to transform it to $(0, \ldots, 0, |x_{max}|, 0, \ldots, 0)$. Recall that the coordinates we are interested in are only the first $|\Sigma|$. Let us call v to the block containing these coordinates. Since $\text{relu}(x)+\text{relu}(-x) = |x|$ for all x, our algorithm does the following:

$$\begin{pmatrix} v \\ \vdots \end{pmatrix} \to \begin{pmatrix} v \\ -v \\ \vdots \end{pmatrix} \to \begin{pmatrix} \text{relu}(v) \\ \text{relu}(-v) \\ \vdots \end{pmatrix} \to \begin{pmatrix} \text{relu}(v) + \text{relu}(-v) \\ \vdots \end{pmatrix}$$

where relu is applied position wise.

The first step is done in the same way as the first half of the linear transformations. For the second step, we take max-coordinate (Section 3.1.6) in v and $-v$ independently. This works since all the coordinates of v are 0 except one thus there are only two scenarios. The non-zero coordinate of v is positive, then it will be the maximum (so it won't be modified) and all the others will remain 0; or the non-zero coordinate of v is negative, then the maximum will be 0 and all the coordinates will become 0. Whichever happens in v, the other happens in $-v$. Then, for adding both vectors we can use one FF layer with the following parameters. Observe that this works since all the coordinates of $\text{relu}(-v)$ are non-negative.

$$W_1 = id^{d\times d} \qquad W_2 = \begin{pmatrix} 0^{|\Sigma|\times|\Sigma|} & id^{|\Sigma|\times|\Sigma|} & 0^{|\Sigma|\times(d-2|\Sigma|)} \\ 0^{d-|\Sigma|\times|\Sigma|} & 0^{d-|\Sigma|\times|\Sigma|} & 0^{d-|\Sigma|\times d-2|\Sigma|} \end{pmatrix} \qquad b_1 = b_2 = \left(0^{d\times d}\right)$$

SC hacer step by step esta FF

3.3 Boolean Operations

In this section we prove that $\mathsf{CoT}[T(n), d(n)]$ classes are closed under boolean operations. We show that the complement of a language in $\mathsf{CoT}[T(n), d(n)]$ is in $\mathsf{CoT}[T(n), d(n)]$ and that the disjunction of two language of the same class, stays in the same class.

RR los dibujitos de las dos secciones que siguen están hechos pero estoy viendo en qué formato insertarlos. Los quería exportar como svg pero latex no se lleva muy bien con eso. Probablemente terminen siendo pngs pero tengo que revisar la resolución de las imáganes

3.3.1 Complement of a language

Let $\mathcal{L} \in \mathsf{CoT}[T(n), d(n)]$. Then there is a family of transformers $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ of embedding size $d(n)$ that after $T(n)$ steps of CoT prints q_{accept} or q_{reject} depending on if the input word is in the language. For each transformer in the family, we construct a corresponding transformer that behaves identically (internally and externally) but prints the flipped decision token as represented in Figure 3.2.

Let TF be a transformer, we construct TF' that flips the decision token of TF, i.e., if $\mathsf{TF}^*(\alpha) = q_{accept}$, then $\mathsf{TF}'^*(\alpha) = q_{reject}$ and if $\mathsf{TF}^*(\alpha) = q_{reject}$, then $\mathsf{TF}'^*(\alpha) = q_{accept}$. This is the only change made to TF, thus $\mathsf{TF}'^i(\sigma) = \mathsf{TF}^i(\sigma)$ for every i when σ is not a decision symbol.

This is done by swapping the rows associated with q_{accept} and q_{reject} in the matrix of the OUTPUT layer of TF. Thus, when the largest probability is associated with q_{accept} in TF, that same number will be associated to q_{reject} in TF'.

Formally, TF' is defined exactly the same as TF except in the OUTPUT layer, where the matrix is different. Let M be the matrix of the OUTPUT layer of TF, let i be the position of the token q_{accept} and j the position of q_{reject}, then the matrix of the OUTPUT layer of TF' will be $M' = PM$ where P is the matrix that flips the rows i and j.

$$M'_{kl} = \begin{cases} 1 & \text{if } k = l,\ k \neq i \text{ and } k \neq j \\ 1 & \text{if } k = j \text{ and } l = i \\ 1 & \text{if } k = i \text{ and } l = j \\ 0 & \text{otherwise} \end{cases}$$

Since the OUTPUT matrix is the only difference between TF and TF', they have the same budget and embedding size as TF, therefore, TF' belongs to the same class as TF.

We can define the family $\{\mathsf{TF}'_n\}_{n\in\mathbb{N}}$ where TF'_i is the negation of TF_i constructed as explained in this section. Then $\{\mathsf{TF}'_n\}_{n\in\mathbb{N}}$ computes the complement of $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$. Since for every $i \in \mathbb{N}$, TF'_i has the same budget and embedding size as TF_i, $\bar{\mathcal{L}}$ is in $\mathsf{CoT}[T(n), d(n)]$.

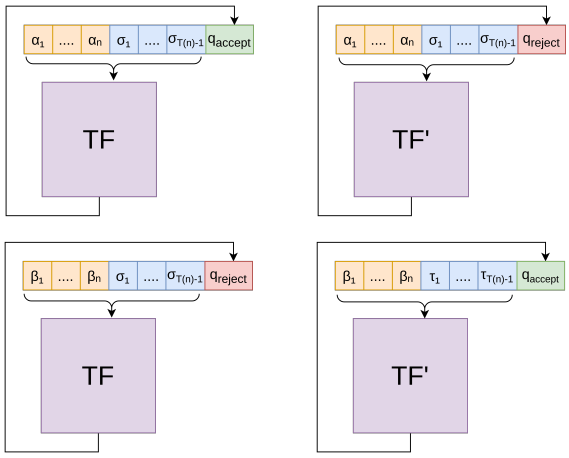

Fig. 3.2: Visual representation of the construction for complement

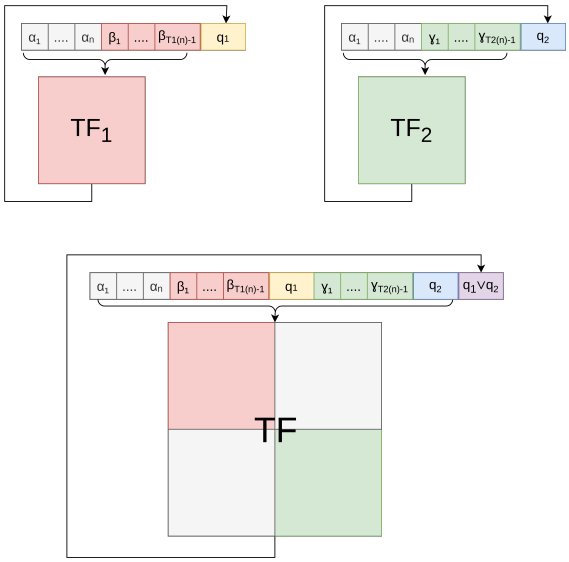

Fig. 3.3: Visual representation of the construction for disjunction

3.3.2 Disjunction of two languages

Let $\mathcal{L}_1$ and $\mathcal{L}_2$ be languages in $\mathsf{CoT}[T(n), d(n)]$, then there is a family of transformers $\{(\mathsf{TF}_1)_n\}_{n\in\mathbb{N}}$ (resp. $\{(\mathsf{TF}_2)_n\}_{n\in\mathbb{N}}$) of embedding size $d_1(n)$ (resp. $d_2(n)$) $\in O(d(n))$ that after $T_1(n)$ (resp. $T_2(n)$) $\in T(n)$ steps of CoT computes $\mathcal{L}_1$ (resp. $\mathcal{L}_2$).

We construct a family $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ that computes $\mathcal{L}_1 \cup \mathcal{L}_2$. It does it in $T_1(n)+T_2(n)+1 \in O(T(n))$ steps of CoT and it has embedding size $O(d(n))$.

The idea behind the construction is to run first one of the transformers, then run the other and finally compute the disjunction of both results as represented in Figure 3.3.

For every input α, the tape of the transformer, at the end of the computation, ends up looking as $\alpha\beta q$, where q is the decision token printed by the transformer and β ($\in \Sigma^*$) is the sequence of intermediate tokens printed during the chain of thought. We denote by $\alpha\beta q_1$ the content of the tape of TF_1 at the end of the computation when the input word is α. Similarly, we denote by $\alpha\gamma q_2$ the content of the tape of TF_2 at the end of the

computation when the input word is α. The tape of TF, computing the disjunction of TF_1 and TF_2, at the end of the computation has content $\alpha\beta q_1 \gamma q_2 q$, where q is the decision token representing the disjunction of q_1 and q_2.

Let us construct TF from TF_1 and TF_2. Since we cannot change the body of the transformer during the iterations, we are required to use some tricks.

First, we grow the dimension of the embedding for running the body of both transformers at the same time over the same vectors but independently. In the final layers we choose which probability distribution will go to the output layer, this makes TF print the token generated by TF_1 or TF_2. As always, we assume that the transformers are normalized.

We use the first half of the embedding for the transformer TF_1 and the second for the transformer TF_2. We have some extra coordinates for flags at the end, but we will ignore them for now.

$$\mathsf{TE}(\sigma)_i = \begin{cases} \mathsf{TE}_1(\sigma)_i & \text{if } i \leq d_1(|\alpha|) \\ \mathsf{TE}_2(\sigma)_{i-d_1(|\alpha|)} & \text{if } d_1(|\alpha|) < i \leq d_2(|\alpha|) \\ 0 & \text{if } d_2(|\alpha|) < i \end{cases}$$

$$\mathsf{PE}(n)_i = \begin{cases} \mathsf{PE}_1(n)_i & \text{if } i \leq d_1(|\alpha|) \\ \mathsf{PE}_2(n)_{i-d_1(|\alpha|)} & \text{if } d_1(|\alpha|) < i \leq d_2(|\alpha|) \\ \text{flags} & \text{if } d_2(|\alpha|) < i \end{cases}$$

RR Me gustaría en vez de poner PE_2 poner algo que se entienda que es más o menos PE_2. Formalmente PE_2 está mal porque no está definida en el rango que se la usa. Debería ser shiftedPE_2 que lo defino más adelante. Hacer esa explicación ahora me parece que mete mucho ruido.

The transformer TF has one layer for each layer of TF_1 and each of TF_2. The first layers correspond to the body of TF_1. They are constructed ignoring the half of the embedding storing the coordinates used by TF_2. The second half of layers are the other way around, they compute the layers of TF_2 and ignore the TF_1 coordinates. Then, all the matrices in the layers of the first part of the body of TF look like V_1 and the ones in the second layers will look like V_2 where W_1 and W_2 are the matrices of the corresponding layers in TF_1 and TF_2.

$$V_1 = \begin{pmatrix} W_1 & 0 \\ 0 & 0 \end{pmatrix} \qquad\qquad V_2 = \begin{pmatrix} 0 & 0 \\ 0 & W_2 \end{pmatrix}$$

It is important to notice that the values of the coordinates corresponding to one transformer do not affect the values of the other. Thus, if the tape of the transformer has content γ, then at the end of the body of TF, the last vector would look like (x, y, flags) where x is the last vector at the end of the body of TF_1 when its tape content is γ, and y the one at the end of TF_2 when its tape content is γ. Therefore, if we always choose to transform said vector into $(x, 0, \ldots, 0)$, then TF will produce the same tape as TF_1. This works for generating the first $T_1(|\alpha|)$ tokens which are βq_1. Thus, we would like to apply this transformation only to the vectors generating them. Therefore, we need to flag them with the position encoding. Since $h_{|\alpha|+i}$ carries the distribution to output in the $(i+1)$-th step, i.e., the $(|\alpha| + i + 1)$-th token of the tape, the vectors that we need to flag will be $h_{|\alpha|}, h_{|\alpha|+1}, \ldots, h_{|\alpha|+T_1(|\alpha|)-1}$.

After running $T_1(|\alpha|)$ steps of this construction, the tape will be $\alpha\beta q_1$. At this point, we want to start generating γ, i.e., the tape of TF_2. Just changing what we choose before the OUTPUT layer does not work, since what the construction will produce would be $\mathsf{TF}_2(\alpha\beta q_1)$, and we need $\mathsf{TF}_2(\alpha)$.

Now we show how to run TF_2 in this construction ignoring the tokens produced by TF_1. We make TF_2 not attend to the tokens in positions $|\alpha|+1$ to $T_1(|\alpha|)$ (this can be done using the primitive presented in the Section 3.1.4). We also need for them to be invisible to the portion of the position encoding corresponding to TF_2, thus we "shift" it in the following way.

$$\mathsf{PE}(n)_i = \begin{cases} \mathsf{PE}_1(n)_i & \text{if } i \leq d_1(|\alpha|) \\ \text{shifted}\mathsf{PE}_2(n)_{i-d_1(|\alpha|)} & \text{if } d_1(|\alpha|) < i \leq d_2(|\alpha|) \\ \text{flags} & \text{if } d_2(|\alpha|) < i \end{cases}$$

$$\text{shifted}\mathsf{PE}_2(n) = \begin{cases} \mathsf{PE}_2(n) & \text{if } n \leq |\alpha| \\ 0 & \text{if } |\alpha| < n < T_1(|\alpha|) \\ \mathsf{PE}_2(n - T_1(|\alpha|)) & \text{if } T_1(|\alpha|) < n \end{cases}$$

The first case is for the positions occupied by the input, we do not want to change their encoding. The second case is for the positions occupied by the tokens generated by TF_1, we do not care about them. The third case is for the tokens that TF_2 will generate. It should believe that they are in their natural positions, i.e., shifted $T_1(|\alpha|)$ positions left (the positions occupied by the tokens generated by TF_1). We also need to change the flag used for choosing the distribution in the vectors at positions after $|\alpha| + T_1(|\alpha|)$ so TF chooses the distribution corresponding to TF_2 instead of TF_1.

At this point, the construction almost produces the tape of TF_1 followed by the tape of TF_2 except for one token. We need to do something special for the first token generated by TF_2. The distribution corresponding to it is in the vector $h_{|\alpha|}$ but the last vector of the first run computing TF_2 tokens is $h_{|\alpha|+T_1(|\alpha|)}$, the one coming from the last token of TF_1. Therefore, before choosing the distribution, we need to copy the vector $h_{|\alpha|}$ to $h_{|\alpha|+T_1(|\alpha|)}$. Notice that nothing more is needed, since the attention mechanism only attends to vectors to the right, so $h_{|\alpha|}$ is not affected by the vectors coming from the tokens produced by TF_1. This is done with the operation presented in Section 3.1.9.

With this last fix, TF generates the tape of TF_1 followed by the tape of TF_2. We now show how to compute q, i.e., the disjunction of q_1 and q_2. This is easily done making $h_{|\alpha|+T_1(|\alpha|)+T_2(\alpha)}$ only attend to itself and to $h_{|\alpha|+T_1(|\alpha|)}$, i.e., to the vectors coming from q_1 and q_2.

We add a flag in the token embedding of TF to recognize the decision tokens of TF_1 and TF_2. The flag is in the coordinate f_c.

$$\mathsf{TE}(\sigma)_{f_c} = \begin{cases} 1 & \text{if } \sigma = q_{accept} \\ 0 & \text{otherwise} \end{cases}$$

Then, adding $h_{|\alpha|+T_1(|\alpha|)}$ to $h_{|\alpha|+T_1(|\alpha|)+T_2(\alpha)}$, we have 1 or 2 in the coordinate f_c iff TF_1 or TF_2 accepted α. Now we make all of $h_{|\alpha|+T_1(|\alpha|)+T_2(\alpha)}$ 0, except the f_c coordinate (using Section 3.1.3).

Before going to the OUTPUT layer we move the value of f_c to the coordinate that will project to the probability of q_{accept} and add some number larger than 0 but smaller than

1 to the coordinate that will define the probability of q_{reject}. Therefore, the token with the maximum probability will be q_{accept} iff TF_1 or TF_2 accepted α, and it will be q_{reject} in the other case.

Then, TF computes the union of the languages of TF_1 and TF_2. The number of steps of CoT that it needs is exactly the ones made by TF_1 plus the ones made by TF_2 plus 1 extra for taking the disjunction, thus in $O(\cdot)$ notation it takes the same as the maximum of TF_1 and TF_2. The embedding dimension of TF is the same as the dimension of TF_1 plus the dimension of TF_2 plus some constant number of flags, thus in $O(\cdot)$ notation it is the same as the maximum of TF_1 and TF_2.

Therefore, the language computed by the family of transformers $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ is in $\mathsf{CoT}[T(n), d(n)]$. Thus, $\mathcal{L}1 \cup \mathcal{L}2 \in \mathsf{CoT}[T(n), d(n)]$.

4. MAIN RESULTS

In this chapter, we present our results on probabilistic transformers. We show that probabilistic transformers are more powerful than deterministic ones, as they are capable of simulating them. We prove that the probabilistic classes support error reduction by means of a Monte Carlo simulation. Furthermore, we also demonstrate that RCoT classes are closed under boolean operations.

Finally, we prove upper and lower bounds on the expressive power of probabilistic transformers. These theorems are extensions of [5]'s results for deterministic transformers.

4.1 Probabilistic can simulate det

RR: no estoy seguro cómo llamar a esta sección

In this section we show how to simulate a deterministic transformer with a probabilistic one. Thus, we obtain the following theorem.

Theorem 4.1.1. $\mathsf{CoT}[T(n), d(n)] \subseteq \mathsf{RCoT}[T(n), d(n)]$ *for all* $T(\cdot)$ *and* $d(\cdot)$

Let TF be a deterministic transformer. We construct a probabilistic transformer TF' that accepts the same words in the same number of steps of CoT and with the same embedding size.

As per Section 3.2, we assume TF to be normalized. We define TF' exactly as TF, except for the token selector which by definition is the probabilistic. Now, TF and TF' behave exactly the same. Since TF is normalized, the distribution p generated in every step satisfies the property that there exists $\sigma \in \Sigma$ such that:

$$p(\tau) = \begin{cases} 1 & \text{if } \tau = \sigma \\ 0 & \text{otherwise} \end{cases}$$

Then, sampling this distribution p always returns the token σ, the same token that argmax would return. Next, the deterministic token selector and the probabilistic token selector behave the same. Thus, for all i and α, $\mathsf{TF}^i(\alpha) = \mathsf{TF}'^i(\alpha)$. In particular, this means that $\mathsf{TF}^*(\alpha) = \mathsf{TF}'^*(\alpha)$. Therefore, TF and TF' accept the same words (and in the same number of steps). By construction, the embedding size of TF and TF' is the same.

Finally, let $\mathcal{L} \in \mathsf{CoT}[T(n), d(n)]$. Then, there is a family of deterministic transformers $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ of embedding size $O(d(n))$ that after $O(T(n))$ steps of CoT computes $\mathcal{L}$.

We define the family $\{\mathsf{TF}'_n\}_{n\in\mathbb{N}}$ of probabilistic transformers by applying the construction just presented to each TF_n. Then, the transformers in $\{\mathsf{TF}'_n\}_{n\in\mathbb{N}}$ have embedding size $O(d(n))$ and compute $\mathcal{L}$ using $O(T(n))$ steps of CoT.

Therefore, $\mathcal{L} \in \mathsf{RCoT}[T(n), d(n)]$, and consequently $\mathsf{CoT}[T(n), d(n)] \subseteq \mathsf{RCoT}[T(n), d(n)]$.

4.2 Normalization

Recall the definition of normalization for deterministic transformers. We define a deterministic transformer as normalized when it satisfies the conditions $\mathsf{N1}$ and $\mathsf{N2}$.

If we make a probabilistic transformer satisfy N2, then, as seen in Section 4.1, it becomes deterministic. This changes the behavior of the transformer, so we cannot claim that for every probabilistic transformer there exists another with the same characteristics that computes the same language and satisfies N2. hus, for probabilistic transformers, we define normalization solely as the condition N1.

Definition 4.2.1 (Normalized probabilistic transformer)**.** We say a probabilistic transformer is normalized if it satisfies the following condition.

- N1: The matrix of the OUTPUT has to be a projection, i.e., it is a diagonal matrix with ones on its diagonal.

Observe that the algorithm presented in Section 3.2.1 for constructing equivalent transformers satisfying N1 in the deterministic setting also applies to probabilistic transformers.

4.3 Robustness of our Definition

Recall the definition of language recognition by probabilistic transformers. We say that a family of probabilistic transformers $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ recognizes a language $\mathcal{L}$ when

$$\alpha \in \mathcal{L} \iff \Pr[\mathsf{TF}^*_{|\alpha|}(\alpha) = q_{accept}] > 2/3$$

The choice of the constant $2/3$ may seem somewhat arbitrary. We now show that we can replace $2/3$ with any constant in the interval $[1/2, 1)$.

Let $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ be a family of probabilistic transformers such that $\alpha \in \mathcal{L} \iff \Pr[\mathsf{TF}^*_{|\alpha|}(\alpha) = q_{accept}] > c$ for some fixed c in the interval $[1/2, 1)$ and show that is equivalent to a family $\{\mathsf{TF}'_n\}_{n\in\mathbb{N}}$ that answers correctly with probability $d < 1$.

For each TF_n we construct TF'_n such that for all words of length n, $\alpha \in \mathcal{L} \iff \Pr[\mathsf{TF}'^*_n(\alpha) = q_{accept}] > d$. First, if $d \leq c$, then $\mathsf{TF}_n = \mathsf{TF}'_n$. Let us explore the case where $c < d$.

We construct TF'_n based on TF_n using a Monte Carlo simulation. The idea is similar to the disjunction of languages. TF'_n runs the transformer TF_n sequentially r times and then analyzes whether the majority of the results accept.

The construction presented in Section 3.3.2 for the disjunction of two deterministic transformers can be easily extended to run a larger number of transformers sequentially. This construction also applies for probabilistic transformers.

To compute the disjunction, we analyzed the coordinate f_c at the last step of the computation. It worked as a counter of how many transformers accepted the word. We replicate this strategy to check whether the majority of the r runs of TF_n accepted the word, i.e., $f_c > r/2$. If that is the case, we force TF'_n to accept; otherwise, we force it to reject.

The question is how many runs of TF_n are needed, given that it answers correctly with probability $c > 1/2$, in order to obtain the correct answer with probability at least d. We estimate the necessary r using Chernoff's bound.

Theorem 4.3.1 (Chernoff's bound)**.** *Let $y_1, \ldots, y_r$ be independent Boolean random variables with $\Pr[y_i = 1] = c$ for every $1 \leq i \leq r$. Let $Y = \sum_{i=1}^{r} y_i$ and let $\delta \in (0, 1)$. Then $\Pr[Y \leq (1-\delta)rc] \leq e^{-\frac{\delta^2}{4}rc}$ and $\Pr[Y \geq (1+\delta)rc] \geq e^{-\frac{\delta^2}{4}rc}$.*

We need to find δ and large enough r such that $\Pr[Y > r/2] > d$. Let us analyze how the bound applies to our case. Each y_i represents a run of TF_n. $y_i = 1$ indicates that the ith run of TF_n answered correctly, and $y_i = 0$ that it answered incorrectly. Since TF_n answers correctly with probability higher than c, we have $\Pr[y_i = 1] > c$.

Applying Chernoff's bound, we require δ and r such that $(1+\delta)rc > r/2$ and $e^{-\delta^2 rc/4} \geq d$.

$$\begin{aligned} (1+\delta)rc &> \frac{r}{2} & e^{-\frac{\delta^2}{4}rc} &\geq d \\ (1+\delta)c &> \frac{1}{2} & -\frac{\delta^2}{4}rc &\geq \log_e(d) \\ c + \delta c &> \frac{1}{2} & r &\geq -4\frac{\log_e(d)}{c\delta^2} \\ \delta &> \frac{1/2 - c}{c} \end{aligned}$$

Since $c \geq 1/2$, we can bound $(1/2 - c)/c$ by 0; thus, choosing any $\delta \in (0, 1)$ satisfies the first equation. Then, with a fixed δ, choosing $r = -4\log_e(d)/(c\delta^2)$ makes $\Pr[Y > r/2] > d$. Therefore, TF'_n runs TF_n $\lceil -4\log_e(d)/(c\delta^2) \rceil$ times and accepts the word if more than half of the runs accepted; otherwise, it rejects. Finally, TF'_n answers correctly with probability at least d.

For $|\alpha| = n$, let $T(n)$ be the number of steps the family $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ needs to compute $\mathcal{L}(\alpha)$ and $d(n)$ its embedding dimension. Then, the family $\{\mathsf{TF}'_n\}_{n\in\mathbb{N}}$ takes $O(r \cdot T(n))$ steps to compute $\mathcal{L}(\alpha)$ and uses embedding size $O(r \cdot d(n))$. Thus, the $\mathsf{RCoT}[T(n), d(n)]$ classes are invariant to the choice of the constant in the definition of language recognition by probabilistic transformers.

4.4 Closed by Boolean Operations

We showed in Section 3.3 that deterministic transformers can perform boolean operations with only a constant overhead; thus, $\mathsf{CoT}[T(n), d(n)]$ classes are closed under boolean operations. Similar constructions apply to probabilistic transformers. Therefore, we show that $\mathsf{RCoT}[T(n), d(n)]$ classes are closed under complement and disjunction, which together imply closure under boolean operations.

Theorem 4.4.1 ($\mathsf{RCoT}[T(n), d(n)]$ is closed under complement)**.** $\mathcal{L} \in \mathsf{RCoT}[T(n), d(n)] \implies \mathcal{L}^c \in \mathsf{RCoT}[T(n), d(n)]$

Let $\mathcal{L} \in \mathsf{RCoT}[T(n), d(n)]$ and let $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ be the family of probabilistic transformers that computes $\mathcal{L}$. Let us assume all the transformers in the family are normalized. Then, similarly to the deterministic case, we define the family $\{\mathsf{TF}'_n\}_{n\in\mathbb{N}}$ as $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$, except the OUTPUT matrices have the rows corresponding to q_{accept} and q_{reject} swapped.

Next, for all $\alpha \in \Sigma^*$, if $\Pr[\mathsf{TF}^*_{|\alpha|}(\alpha) = q_{accept}] > 2/3$, then $\Pr[\mathsf{TF}'^*_{|\alpha|}(\alpha) = q_{reject}] > 2/3$, since every time $\mathsf{TF}_{|\alpha|}$ prints q_{accept}, $\mathsf{TF}'_{|\alpha|}$ prints q_{reject}. Therefore, $\{\mathsf{TF}'_n\}_{n\in\mathbb{N}}$ computes $\mathcal{L}^c$ using the same number of CoT steps and the same embedding size $O(d(n))$ as $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ to compute $\mathcal{L}$. Thus, $\mathcal{L} \in \mathsf{RCoT}[T(n), d(n)]$ implies $\mathcal{L}^c \in \mathsf{RCoT}[T(n), d(n)]$.

Theorem 4.4.2 ($\mathsf{RCoT}[T(n), d(n)]$ is closed under disjunction)**.** $\mathcal{L}_1, \mathcal{L}_2 \in \mathsf{RCoT}[T(n), d(n)] \implies \mathcal{L}_1 \cup \mathcal{L}_2 \in \mathsf{RCoT}[T(n), d(n)]$

Let $\{\mathsf{TF}^1_n\}_{n\in\mathbb{N}}$ be the family of probabilistic transformers recognizing $\mathcal{L}_1$ and $\{\mathsf{TF}^2_n\}_{n\in\mathbb{N}}$ the one recognizing $\mathcal{L}_2$. We construct the family $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ following the same construction as for the disjunction of deterministic transformers (Section 3.3.2). By the result of Section 4.3, we can assume that the probability of error of each family is arbitrary small. Let p_1 be the probability of $\{\mathsf{TF}^1_n\}_{n\in\mathbb{N}}$ answering correctly and p_2 the probability of $\{\mathsf{TF}^2_n\}_{n\in\mathbb{N}}$ answering correctly. Then, for each word α, we have four possible cases. In Figure 4.1, we show the probability of TF^1 and TF^2 accepting or rejecting α in each of the cases.

	TF^1 accepts	TF^1 rejects
TF^2 accepts	p_1p_2	$(1-p_1)p_2$
TF^2 rejects	$p_1(1-p_2)$	$(1-p_1)(1-p_2)$

Case 1: $\alpha \in \mathcal{L}_1 \wedge \alpha \in \mathcal{L}_2$

	TF^1 accepts	TF^1 rejects
TF^2 accepts	$p_1(1-p_2)$	$(1-p_1)(1-p_2)$
TF^2 rejects	p_1p_2	$(1-p_1)p_2$

Case 2: $\alpha \in \mathcal{L}_1 \wedge \alpha \notin \mathcal{L}_2$

	TF^1 accepts	TF^1 rejects
TF^2 accepts	$(1-p_1)p_2$	p_1p_2
TF^2 rejects	$(1-p_1)(1-p_2)$	$p_1(1-p_2)$

Case 3: $\alpha \notin \mathcal{L}_1 \wedge \alpha \in \mathcal{L}_2$

	TF^1 accepts	TF^1 rejects
TF^2 accepts	$(1-p_1)(1-p_2)$	$p_1(1-p_2)$
TF^2 rejects	$(1-p_1)p_2$	p_1p_2

Case 4: $\alpha \notin \mathcal{L}_1 \wedge \alpha \notin \mathcal{L}_2$

Fig. 4.1: Probabilities of all possible behaviors of TF^1 and TF^2

RR: ya sé que estas tablas se ven mal. Tengo que revisarles el estilo

TF accepts whenever TF^1 or TF^2 accepts. Let us analyze case by case the probability of TF answering correctly.

- Case 1: $p_1 + p_2 - p_1p_2 = p_1p_2 + (1-p_1)p_2 + p_1(1-p_2)$
- Case 2: $1 - p_2 + p_1p_2 = p_1(1-p_2) + (1-p_1)(1-p_2) + p_1p_2$
- Case 3: $1 - p_1 + p_1p_2 = (1-p_1)p_2 + p_1p_2 + (1-p_1)(1-p_2)$
- Case 4: p_1p_2

Then, if $p_1 = p_2 = 4/5$, the probability of TF answering correctly in each case is larger than $1/2$.

- Case 1: $24/25 > 1/2$
- Case 2: $21/25 > 1/2$
- Case 3: $21/25 > 1/2$
- Case 4: $16/25 > 1/2$

Thus, the probability of $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ answering $\mathcal{L}_1 \cup \mathcal{L}_2$ correctly is at least $1/2$. Therefore, by the construction in Section 4.3, there is a family of transformers in $\mathsf{RCoT}[T(n), d(n)]$ that recognizes $\mathcal{L}_1 \cup \mathcal{L}_2$ in $\mathsf{RCoT}[T(n), d(n)]$.

4.5 Upper Bound

In this section, we show that one forward pass of constant-depth, polynomial-embedding-dimension, and constant-precision probabilistic transformers can be simulated by shallow Boolean circuits.

Theorem 4.5.1. $\mathsf{RCoT}[0, \mathsf{poly}(n)] \subseteq \mathsf{BPAC}^0[1] = \mathsf{AC}^0$

For this proof, we build upon [5]'s proof of Theorem 2.5.1. They already showed that the distribution generator can be simulated by AC^0 circuits. Thus, we only show that the probabilistic token selector can be simulated by AC^0 circuits.

We can sample from the distribution p by drawing a uniform random variable on the interval $[0, 1)$. We refer to the value of this variable as the seed. This is because we can partition $[0, 1)$ into disjoint intervals, one per token, each of length equal to the token's probability. Let p be the probability computed by the distribution generator. Let $\Sigma = \{\sigma_1, \ldots, \sigma_{|\Sigma|}\}$. Then the ranges are defined as $\text{range}(\sigma_i) = [\sum_{j=1}^{i-1} p(\sigma_j), \sum_{j=1}^{i} p(\sigma_j))$.

For example, if $\Sigma = \{t_1, t_2, t_3\}$ and p is such that $p(t_1) = 0.33$, $p(t_2) = 0.22$, $p(t_3) = 0.45$, then the ranges would be $[0, 0.33)$ for t_1, $[0.33, 0.55)$ for t_2, and $[0.55, 1)$ for t_3.

Observe that although the numbers $p(\sigma_j)$ are representable in $\mathbb{F}_{e,s}$, the summations are not performed with iterative rounding. They are computed with the maximum precision needed to avoid rounding error; this precision can be bounded as a function of e and s alone, independently of the values of $p(\sigma_i)$.

Since the precision for the numbers is fixed and the number of terms of the summations are bounded by the size of the alphabet, the ranges can be computed by circuits in AC^0.

RR: quizás agregar ref a seccion 2.4

Let us analyze the seed. It is just a random number in the range $[0, 1)$. We can interpret it as a binary string. How many bits of the seed are needed to determine which range it belongs to? It suffices to examine as many bits of the seed as the logarithm of the smallest probability.

Since p comes from the distribution generator, for all tokens, $p(\sigma)$ has to be representable in $\mathbb{F}_{e,s}$. Then, the smallest non-empty range has size at least ϵ, where ϵ is the smallest positive number representable in $\mathbb{F}_{e,s}$. Let r be the number of bits needed for representing ϵ in binary. Therefore, knowing the first r bits of the binary representation of the seed is enough to sample p. Thus, we only need r random bits for sampling p.

Note that r only depends on the sizes (e and s) of the significand and exponent of $\mathbb{F}_{e,s}$. Thus, r is constant for constant-precision transformers.

Observe that once the ranges are computed, they act as a lookup table with one entry for each number in $[0, 1)$ whose binary representation has r bits. Since r is a constant, the ranges can be interpreted as a constant-size lookup table.

Therefore, since the ranges can be computed by AC^0 circuits and the lookup table has constant size, sampling from p can be performed by a circuit in $\mathsf{BPAC}^0[1]$.

Thus, by Theorem 2.3.3, one forward pass of constant-depth, polynomial-embedding-dimension, and constant-precision probabilistic transformers can be simulated by a circuit in AC^0.

As in Corollary 2.5.1, we now prove that a logarithm number of CoT steps are not sufficient to go beyond AC^0, even in a probabilistic transformer.

Since we need a constant number of bits from the seed to simulate each CoT step, we need $O(\log(n))$ random bits to simulate $\log(n)$ steps. Thus, we can enumerate all possible

chain-of-thought token sequences ($|\Sigma|^{O(log(n))} = O(n)$) and seeds $2^{O(\log(n))=O(n)}$ and check when they are consistent. Observe that we need to check each seed with each sequence; thus, we check consistency of $O(n^2)$ pairs (sequence, seed).

In the deterministic transformers it was enough to perform an OR of all the consistent sequences. In the probabilistic setting, that idea does not work, since there could be false positives. Instead, our circuit computes $\mathsf{MAJORITY}$ of the consistent runs and outputs its result.

The circuit is clearly in AC^0, except for the $\mathsf{MAJORITY}$. The $\mathsf{MAJORITY}$ problem can be solved in AC^0 only when there is a promise that more than a fraction c of the bits are 1s (or 0s), for a fixed $c \in (1/2, 1]$. Since the probabilistic transformer outputs the correct answer $2/3$ of the time, i.e., for $2/3$ of the seeds, we know that at least $2/3$ of the consistent runs agree. Thus, instead of $\mathsf{MAJORITY}$, we compute $\mathsf{MAJORITY}_{2/3}$, which can be done in AC^0.

Finally, since there is an AC^0 circuit simulating one forward pass of constant-depth, polynomial-embedding-dimension, and constant-precision probabilistic transformers (Theorem 4.5.1), there is an AC^0 circuit simulating $O(\log(n))$ steps of CoT of the same set-up.

Corollary 4.5.1. $\mathsf{RCoT}[\log(n), \mathsf{poly}(n)] \subseteq \mathsf{AC}^0$

Since one forward pass of constant-depth, polynomial-embedding-dimension, and constant-precision probabilistic transformers can be simulated by a circuit in BPAC^0, stacking the circuits of $\mathsf{BPAC}^0[1]$ for each CoT step gives a simulation for $\mathsf{poly}(n)$ steps (as in Corollary 2.5.2). This circuit has polynomial size and uses a polynomial number of random bits; thus, we obtain Corollary 4.5.2.

Corollary 4.5.2. $\mathsf{RCoT}[\mathsf{poly}(n), \mathsf{poly}(n)] \subseteq \mathsf{BPP/Poly}$

4.6 Lower Bound

Building upon Theorem 2.5.2, we prove that constant-depth, constant-precision probabilistic transformers using T steps of CoT with $O(\log(n))$ embedding size can simulate a randomized circuit of size T.

Theorem 4.6.1. *For any polynomial function* $T : \mathbb{N}^+ \to \mathbb{N}^+$, $\mathsf{BPSIZE}(T(n)) \subseteq \mathsf{RCoT}[T(n), \log(n)]$. *In particular,* $\mathsf{BPP/Poly} \subseteq \mathsf{RCoT}[\mathsf{poly}(n), \log(n)]$.

Let $\{C_n\}_{n\in\mathbb{N}}$ be a family of randomized circuits of polynomial size. We construct a family of probabilistic transformers $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ such that each TF_n simulates C_n.

As mentioned in Section 2.3, a randomized circuit C_n accepts α when $\Pr_{r_1 \ldots r_{f(n)}}[C'_n(\alpha, r) = 1] > 2/3$, that is, the deterministic circuit C'_n accepts (α, r) for $2/3$ of the possible values of r.

Our construction first generates r at random and then simulates the deterministic circuit using the construction of [5] for Theorem 2.5.2.

Let TF'_n be the deterministic transformer simulating C'_n in $|C'_n|$ steps of CoT. Then, $\mathsf{TF}'_n(\alpha r) = C'_n(\alpha, r)$.

We assume TF'_n to be normalized. Then, its OUTPUT matrix is a projection.

TF'_n computes the i-th gate when the tape contains $i-1$ elements, i.e., when the values of all gates with lower index have been computed. It computes C'_n starting by the gate

with index $n + f(n) + 1$. By our assignment of the indeces, the first Boolean gate is the one with index $n + f(n) + 1$. Thus, TF'_n begins meaningful computation once its tape contains $n + f(n)$ printed tokens.

We construct TF_n that simulates C_n by modifying TF'_n. Observe that since TF'_n is normalized, the change in token selector from a deterministic to a probabilistic does not affect its behavior. We define TF_n exactly as the probabilistic version of TF'_n, with one modification: it ignores the distributions generated by TF'_n in the first $f(n)$ steps of CoT.

The value in each position in the tape of TF_n corresponds to the value of a gate in C_n. The positions may hold the value 1, the value 0, or remain empty if their value has not yet been computed. The first n positions are the bits of α. The following $f(n)$ are the bits of r. TF'_n can generate the tokens for the positions after $f(n)$, i.e., compute the values of the boolean gates. The probability distributions TF'_n generates in the first $f(n)$ steps, i.e., the distributions corresponding to the tokens representing r, do not correspond to what we want (since from TF'_n's perspective, they are part of the input."). Thus, using the PE, we flag positions n through $n + f(n) - 1$ so that a constant function can be applied to them at the end of the body, forcing the probability distribution for tokens at positions $n + 1$ through $n + f(n)$ to be the uniform distribution over $\{0, 1\}$.

Then, after $f(n)$ steps of CoT, in the tape of TF_n there is written α followed by $f(n)$ random bits. From this point onward, TF'_n simulates $C'_n(\alpha, r)$.

TF_n accepts α if it accepts for at least $2/3$ of the possible runs. TF_n uses randomness only to generate the first $f(n)$ tokens. It generates them following a uniform distribution. Then, $\Pr[\mathsf{TF}_n(\alpha) = 1] = \Pr_r[\mathsf{TF}'_n(\alpha r) = 1] = \Pr_r[C'(\alpha, r) = 1] = \Pr[C_n(\alpha) = 1]$. Therefore, $\Pr[\mathsf{TF}_n(\alpha) = 1] > 2/3$ if and only if $\Pr[C_n(\alpha) = 1] > 2/3$.

4.7 Relations between models

4.7.1 Non-uniform transformers and circuits

From Corollary 2.5.2 and Theorem 2.5.2, we deduce the followin theorem:

Theorem 4.7.1. $\mathsf{CoT}[\mathsf{poly}(n), \mathsf{poly}(n)] = \mathsf{P/Poly}$.

In the same way, from Corollary 4.5.2 and Theorem 4.6.1, it follows Theorem 4.7.2.

Theorem 4.7.2. $\mathsf{RCoT}[\mathsf{poly}(n), \mathsf{poly}(n)] = \mathsf{BPP/Poly}$.

Then, from Theorems 4.7.1 and 4.7.2 its deduced the following relation.

Theorem 4.7.3. $\mathsf{P/Poly} = \mathsf{BPP/Poly} \iff \mathsf{CoT}[\mathsf{poly}(n), \mathsf{poly}(n)] = \mathsf{RCoT}[\mathsf{poly}(n), \mathsf{poly}(n)]$.

This means that randomness boosts the power of transformers if and only if it also does for polynomial-size circuits, which is not known, as probabilistic circuits have not been deeply studied.

4.7.2 Uniform transformers and Turing machines

A more interesting question arises when we require the transformers to be efficiently constructible.

Observe that the proof of Theorem 2.5.2 yields a polynomial algorithm that, for each circuit, constructs its corresponding deterministic transformer. This motivates the definition of $(\mathsf{Uniform})\mathsf{CoT}[T(n), d(n)]$, which is $\mathsf{CoT}[T(n), d(n)]$ restricted to families of transformers such that there exists a deterministic Turing machine that runs in polynomial time and, given the input length 1^n, outputs the transformer TF_n. Since $\mathsf{P/Poly}$ restricted to uniform circuits coincides with P , we obtain the following corollary.

RR: agregar ref a **??**

Corollary 4.7.1. $\mathsf{P} \subseteq (\mathsf{Uniform})\mathsf{CoT}[\mathsf{poly}(n), \mathsf{poly}(n)]$

Observe that Corollary 4.7.1 and Corollary 2.5.2 together imply Theorem 4.7.4.

Theorem 4.7.4. $\mathsf{P} = (\mathsf{Uniform})\mathsf{CoT}[\mathsf{poly}(n), \mathsf{poly}(n)]$

These results have their analogous in the probabilistic setting. In the same way as in the proof of Theorem 2.5.2, the proof of Theorem 4.6.1 yields a polynomial algorithm that, for each randomized circuit, constructs its corresponding probabilistic transformer. Next, we define $(\mathsf{Uniform})\mathsf{RCoT}[T(n), d(n)]$, which is $\mathsf{RCoT}[T(n), d(n)]$ restricted to families of transformers such that there exists a deterministic Turing machine that runs in polynomial time and, given the input length 1^n, outputs the transformer TF_n. Since $\mathsf{BPP/Poly}$ restricted to uniform circuits coincides with BPP , we obtain the following corollary.

RR: agregar ref a **??**

Corollary 4.7.2. $\mathsf{BPP} \subseteq (\mathsf{Uniform})\mathsf{RCoT}[\mathsf{poly}(n), \mathsf{poly}(n)]$

Observe that Corollaries 4.7.2 and 4.5.2 together imply the following theorem.

Theorem 4.7.5. $\mathsf{BPP} = (\mathsf{Uniform})\mathsf{RCoT}[\mathsf{poly}(n), \mathsf{poly}(n)]$

We deduce the uniform version of Theorem 4.7.3.

Theorem 4.7.6. $\mathsf{P} = \mathsf{BPP} \iff (\mathsf{Uniform})\mathsf{CoT}[\mathsf{poly}(n), \mathsf{poly}(n)] = (\mathsf{Uniform})\mathsf{RCoT}[\mathsf{poly}(n), \mathsf{poly}(n)]$.

Although $\mathsf{P} = \mathsf{BPP}$ is an open question in complexity theory, this theorem is an interesting result. It implies that, when restricted to uniform families of transformers, the constrained access to randomness that transformers have is equivalent in expressive power to the unrestricted access allowed to probabilistic polynomial-time Turing machines.

4.7.3 Uniform and non-uniform transformers

We show that non-uniformity in deterministic transformers can simulate randomness in uniform probabilistic transformers. Theorems 4.7.5 (a), Adleman (b) and 4.7.1 (c) give the following relation:

$$(\mathsf{Uniform})\mathsf{RCoT}[\mathsf{poly}(n), \mathsf{poly}(n)] \overset{(a)}{=} \mathsf{BPP} \overset{(b)}{\subseteq} \mathsf{P/Poly} \overset{(c)}{=} (\mathsf{Uniform})\mathsf{CoT}[\mathsf{poly}(n), \mathsf{poly}(n)]$$

Thus, we have Theorem 4.7.7:

Theorem 4.7.7. $(\mathsf{Uniform})\mathsf{RCoT}[\mathsf{poly}(n), \mathsf{poly}(n)] \subseteq \mathsf{CoT}[\mathsf{poly}(n), \mathsf{poly}(n)]$

5. CONCLUSIONS

In this work, we studied the expressive power of constant-depth, constant-precision deterministic and probabilistic transformers. We proved that Boolean operations between languages can be computed in both settings without a blow-up in resources (Sections 3.3 and 4.4); and we were able to extend known expressiveness results from a deterministic setting (Section 2.5) to a randomized one (Sections 4.5 and 4.6).

One of the main results of this work relates the expressive power of deterministic and probabilistic transformers, between them and with other models of computation (Section 4.7). Although it is an open question, it is widely believed that $\mathsf{P} = \mathsf{BPP}$; thus, Theorem 4.7.6 seems to indicate that, if we are only interested in uniform transformers, it is enough to study the deterministic ones. A deeper understanding of the classes $\mathsf{P/Poly}$ and $\mathsf{BPP/Poly}$ and their relationship would be necessary to determine whether the same conclusion holds in the non-uniform case (Theorem 4.7.3).

There is no consensus in the community as to whether the best abstraction of real-life transformers is given by uniform or non-uniform families. We agree with Lance Fortnow that it is reasonable to allow non-uniform families. The uniform constraint was artificially introduced to Boolean circuits to make the power of polynomial-size circuits match the power of polynomial-time deterministic Turing machines. This constraint semantically says that there has to be some relation between the circuits of the same family, i.e., there has to be a polynomial-time deterministic Turing machine that computes the family.

RR: cite: https://blog.comp and-power-of-nonuniform-circuits.html

In real life, models undergo computationally intensive training in which they learn their weights. This extensive training is analogous to the power conferred by non-uniformity on polynomial-size circuits, which is generally believed to be slightly greater than that of P Recall that it is widely believed that $\mathsf{P/Poly} \not\subseteq \mathsf{NP}$.

Consistent with the accepted belief that non-uniformity gives just enough power to hide randomness, we proved that allowing non-uniformity in deterministic transformers is sufficient to simulate uniform probabilistic transformers (Theorem 4.7.7). It remains an open question whether deterministic transformers can simulate probabilistic ones without the help of non-uniformity. Determining whether such a simulation is possible, and characterizing its overhead, would provide insight into the relationship between the expressive power of the two models and the practical implications of Theorem 4.7.3.

Theorems 2.5.1 and 4.5.1 show that $O(\log(n))$ steps of CoT are insufficient to separate the expressive power of probabilistic and deterministic transformers, both in the uniform and non-uniform cases. We are confident that their power is the same, since AC^0 should be a tight upper bound for one pass of a transformer. We do not believe the ATTN mechanism can be simulated in $\mathsf{NC}^0 \subsetneq \mathsf{AC}^0$, since it involves a linear number of variables across a constant number of operations.

BIBLIOGRAPHY

[1] A. Bick, A. Blandin, and D. J. Deming. The rapid adoption of generative AI. Management Science, 1 2026.

[2] D. Goldberg. What every computer scientist should know about floating-point arithmetic. ACM computing surveys (CSUR), 23(1):5–48, 1991.

[3] W. Hesse. Division is in uniform tc0. In International Colloquium on Automata, Languages, and Programming, pages 104–114. Springer, 2001.

[4] IEEE. Ieee standard for floating-point arithmetic. IEEE Std 754-2008, pages 1–70, 2008.

[5] Z. Li, H. Liu, D. Zhou, and T. Ma. Chain of thought empowers transformers to solve inherently serial problems. In The Twelfth International Conference on Learning Representations, 2024.

[6] W. Liang, Y. Zhang, Z. Wu, H. Lepp, W. Ji, X. Zhao, H. Cao, S. Liu, S. He, Z. Huang, D. Yang, C. Potts, C. D. Manning, and J. Y. Zou. Mapping the increasing use of llms in scientific papers, 2024.

[7] T. Lin, Y. Wang, X. Liu, and X. Qiu. A survey of transformers. AI Open, 3:111–132, 2022.

[8] A. Madaan and A. Yazdanbakhsh. Text and patterns: For effective chain of thought, it takes two to tango. arXiv preprint arXiv:2209.07686, 2002.

[9] W. Merrill and A. Sabharwal. A little depth goes a long way: The expressive power of log-depth transformers. arXiv preprint arXiv:2503.03961, 2025.

[10] W. Merrill and A. Sabharwal. The parallelism tradeoff: Limitations of log-precision transformers. Transactions of the Association for Computational Linguistics, 11:531–545, 2023.

[11] J. Pérez, J. Marinković, and P. Barceló. On the turing completeness of modern neural network architectures. arXiv preprint arXiv:1901.03429, 2019.

[12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin. Attention is all you need. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, Advances in Neural Information Processing Systems, volume 30. Curran Associates, Inc., 2017.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. Advances in neural information processing systems, 30, 2017.

[14] B. Wang, S. Min, X. Deng, J. Shen, Y. Wu, L. Zettlemoyer, and H. Sun. Towards understanding chain-of-thought prompting: An empirical study of what matters. In A. Rogers, J. Boyd-Graber, and N. Okazaki, editors, Proceedings of the 61st Annual

Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 2717–2739, Toronto, Canada, July 2023. Association for Computational Linguistics.

[15] A. Yang, C. Watson, A. Xue, S. Bhattamishra, J. Llarena, W. Merrill, E. D. S. Ferreira, A. Svete, and D. Chiang. The transformer cookbook, 2025.